

**Meeting and
Scheduling System
Code and Testing Documentation
Version 1.0**

TEAM MEMBERS

Name	Student ID
Samuel Shumake	sms8868
Victoria Worthington	vkW5048
Lianjie Sun	ljs6051
Tiffany Hart	tph5606

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Revision History

Date	Version	Description	Author
08/14/2026	1.0	Coding and Testing	All

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Table of Contents

1.	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms, and Abbreviations	4
1.4	References	4
1.5	Overview	4
2.	Coding and Testing	4
2.1	Code Static Analysis	5
2.1.1	SecurityConfig.java	5
2.1.2	AdminScheduleController.java	5
2.1.3	BillingController.java	5
2.1.4	ComplaintController	5
2.1.5	MeetingController	6
2.1.6	RegisterController	6
2.1.7	RegisterControllerIF	6
2.1.8	RoomController	6
2.1.9	ScheduleController	7
2.1.10	GlobalExceptionHandler	7
2.1.11	Client	7
2.1.12	Complaint	7
2.1.13	Meeting	7
2.1.14	User	8
2.1.15	MeetingRepository	8
2.1.16	MeetingRowMapper	8
2.1.17	UserRepository	8
2.1.18	UserRowMapper	8
2.1.19	LoginSuccessHandler	8
2.1.20	UserService	8
2.2	Code Documentation	9
2.2.1	Class Diagram	9
2.2.2	All Classes and Methods	18
2.3	Test Results	47
2.3.1	Unit and Integration Testing	47
2.3.2	Fuzz Testing	48
2.3.3	Security Testing	48
2.3.4	Network Auditing / Mapping	58
2.3.5	Vulnerability Scanning	59
2.3.6	Bugs	60
	Final Security Review	61
2.4	Passed	61
2.5	Passed with Recommendations	61
2.6	Failed	61
2.7	Conclusion	61
3.	Team Member Log Sheets	61
3.1	Samuel Shumake	61
3.2	Victoria Worthington	62
3.3	Lianjie Sun	62
3.4	Tiffany Hart	63

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Project Analysis and Vision

1. Introduction

This documentation specifies the Coding and Testing (CT) implementation of the development phase for the PennStateSoft Meeting and Scheduling System (PMSS). It will include sections devoted to the Coding and Testing segments of the application. The following sub-sections will cover the purpose, scope, and overview of this document as well as an overview of any definitions and references needed to understand the material.

1.1 Purpose

The purpose of this document is to document the coding and testing completed during the development of the PennStateSoft Meeting Scheduling System (PMSS). It will cover weaknesses and vulnerabilities that were identified and addressed during development, as well as provide an overview of the testing performed and the results of those tests.

1.2 Scope

This document covers the coding and testing completed during the development of the PennStateSoft Meeting Scheduling System (PMSS). It includes the implementation of the system's features, security controls, input validation, authentication and authorization, and other important coding considerations. The testing portion covers the different tests performed to make sure the application works correctly and securely. This includes functionality testing, security and vulnerability testing, and static code analysis. Any bugs, vulnerabilities, or other issues found during the testing are documented along with how they were resolved or why they were left unresolved.

1.3 Definitions, Acronyms, and Abbreviations

Acronym/Abbreviation	Definition
CT	Coding and Testing
PMSS	PennStateSoft Meeting Scheduling System
MVC	Model-View-Controller
JDBC	Java Database Connectivity API

Table 1: Definitions Table

1.4 References

This subsection provides a complete list of documents referenced through this CT:

1. SonarSource. *SonarQube, Version 25.x*. SonarSource, 2026, <https://www.sonarsource.com/products/sonarqube/>.
2. JUnit Team. *JUnit 5, Version 5.x*. JUnit, 2026, <https://junit.org/junit5/>.
3. Nmap Development Team. *Nmap, Version 7.x*. Nmap, 2026, <https://nmap.org/>.
4. Tenable. *Nessus, Version 10.x*. Tenable, 2026, <https://www.tenable.com/products/nessus>.
5. PortSwigger. *Burp Suite, Version 2026.x*. PortSwigger, 2026, <https://portswigger.net/burp>.

1.5 Overview

The document is divided into sections covering the coding and testing performed for PMSS. The coding section discusses the implementation of the application and the coding practices used during development, including security and code quality improvements. The test section describes the tests performed on the application and their results. Issues found during testing are documented along with the steps taken to correct them. Static code analysis results and other testing results are also included to show how potential problems in the application are identified and addressed.

2. Coding and Testing

The coding and testing phase of PMSS focused on making sure the system worked as intended while also addressing security and code quality issues. During development and testing, several bugs, vulnerabilities, and other issues were found and reviewed. Different types of testing were used to check the application, including static code analysis unit testing, integration testing, fuzz testing, network auditing / mapping testing, and vulnerability scanning. Tools such

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

as SonarQube, Nessus, Nmap, JUnit 5, and Burp Suite were used to help identify potential problems. The issues found during testing were then reviewed and fixed when possible, with the results and resolutions documented in the following sections. The coding and testing process helped improve the functionality, security, and reliability of PMSS and provide a way to verify that the system was working as expected.

2.1 Code Static Analysis

Results

Tools: SonarQube for IDE on IntelliJ

Target: PMSS – 192.168.1.166 : 8080

Lines of Code: 1431

Classes with findings: 20

Classes without findings: 18

Issues found: 59

Issues resolved: 55

Issues unresolved: 4

2.1.1 SecurityConfig.java

Location	Issue	Severity	Resolution
(90, 57)	Replace this lambda with the method reference 'SessionFixationConfigurer::changeSessionId'.	Low	Fixed
(38, 32)	Define a constant instead of duplicating this literal "/login" 4 times	High	Fixed

Table 2: SecurityConfig Table

2.1.2 AdminScheduleController.java

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed

Table 3: AdminScheduleController Table

2.1.3 BillingController.java

Location	Issue	Severity	Resolution
(56, 49)	Define a constant instead of duplicating this literal "errorMessage" 3 times	High	Fixed

Table 4: BillingController Table

2.1.4 ComplaintController

Location	Issue	Severity	Resolution
(121, 19)	Define a constant instead of duplicating this literal "redirect:/complaints/list" 4 times	High	Fixed
(68, 19)	Define a constant instead of duplicating this literal "redirect:/complaints" 8 times	High	Fixed
(66,49)	Define a constant instead of duplicating this literal "errorMessage" 4 times	High	Fixed
(52, 27)	Define a constant instead of duplicating this literal "complaints" 4 times	High	Fixed
(87, 45)	Define a constant instead of duplicating this literal "successMessage" 3 times	High	Fixed
(97, 13)	Define a constant instead of duplicating this literal "ADMINISTRATOR" 4 times	High	Fixed

Table 5: ComplaintController Table

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

2.1.5 MeetingController

Location	Issue	Severity	Resolution
(0,0)	Use the "java.time" API for date and time	Info	Fixed
(248, 19)	Define a constant instead of duplicating this literal "redirect:/meetings/" 4 times	High	Fixed
(127, 45)	Define a constant instead of duplicating this literal "successMessage" 5 times	High	Fixed
(87, 31)	Define a constant instead of duplicating this literal "meetingName" 4 times	High	Fixed
(124, 18)	Define a constant instead of duplicating this literal "Meeting \" 4 times	High	Fixed
(91, 31)	Define a constant instead of duplicating this literal "roomNumber" 4 times	High	Fixed
(88, 31)	Define a constant instead of duplicating this literal "meetingDate" 4 times	High	Fixed
(89, 31)	Define a constant instead of duplicating this literal "startTime" 4 times	High	Fixed
(86, 31)	Define a constant instead of duplicating this literal "errorMessage" 6 times	High	Fixed
(46, 15)	Define a constant instead of duplicating this literal "redirect:/schedule" 6 times	High	Fixed
(90, 31)	Define a constant instead of duplicating this literal "endTime" 4 times	High	Fixed
(248, 48)	Define a constant instead of duplicating this literal "/edit" 4 times	High	Fixed

Table 6: MeetingController Table

2.1.6 RegisterController

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed
(115, 65)	Define a constant instead of duplicating this literal "Please enter a valid birth date." 3 times	High	Fixed
(163, 23)	Define a constant instead of duplicating this literal "redirect:/admin/register" 6 times	High	Fixed
(82, 53)	Define a constant instead of duplicating this literal "errorMessage" 10 times	High	Fixed
(84, 23)	Define a constant instead of duplicating this literal "redirect:/register" 5 times	High	Fixed
(260, 19)	Refactor this method to reduce its Cognitive Complexity from 16 to the 15 allowed	High	Allowed – Cognitive Complexity is only one point over recommended complexity limit
(314, 39)	Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method	Info	Fixed

Table 7: RegisterController Table

2.1.7 RegisterControllerIF

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed

Table 8: RegisterControllerIF Table

2.1.8 RoomController

Location	Issue	Severity	Resolution
(53, 19)	Define a constant instead of duplicating this literal "redirect:/admin/rooms" 5 times	High	Fixed

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

(52, 49)	Define a constant instead of duplicating this literal "errorMessage" 3 times	High	Fixed
(64, 22)	Define a constant instead of duplicating this literal "SPECIAL" 3 times	High	Fixed
(60, 20)	Define a constant instead of duplicating this literal "Room " 4 times	High	Fixed
(66, 19)	Extract this nested ternary operation into an independent statement	Medium	Fixed

Table 9: RoomController Table

2.1.9 ScheduleController

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed
(83, 19)	Method has 8 parameters, which is greater than 7 authorized	Medium	Allowed – reducing the number of parameters would require unnecessary restructuring
(83, 19)	Refactor this method to reduce its Cognitive Complexity from 18 to the 15 allowed	High	Allowed – Cognitive Complexity is only three points over recommended complexity limit
(231, 25)	Update this method so that its implementation is not identical to "getUsersMeeting" on line 183	Medium	Fixed

Table 10: ScheduleController Table

2.1.10 GlobalExceptionHandler

Location	Issue	Severity	Resolution
(37, 36)	Invoke method(s) only conditionally	Medium	Fixed
(30, 15)	Define a constant instead of duplicating this literal "error" 3 times	High	Fixed
(29, 27)	Define a constant instead of duplicating this literal "message" 3 times	High	Fixed
(28, 27)	Define a constant instead of duplicating this literal "status" 3 times	High	Fixed

Table 11: GlobalExceptionHandler Table

2.1.11 Client

Location	Issue	Severity	Resolution
(26, 0)	Complete the task associated with this TODO comment	Info	Fixed

Table 12: Client Table

2.1.12 Complaint

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed

Table 13: Complaint Table

2.1.13 Meeting

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed
(34, 8)	Remove this useless assignment; "roomId" already holds the assigned value along all executed paths	Medium	Fixed
(34, 20)	Remove or correct this useless self-assignment	Medium	Fixed
(4, 7)	Remove this unused import 'java.time.LocalDateTime'	Low	Fixed

Table 14: Meeting Table

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

2.1.14 User

Location	Issue	Severity	Resolution
(20, 4)	Change the visibility of this constructor to "protected"	Medium	Fixed
(20, 14)	Constructor has 12 parameters, which is greater than 7 authorized	Medium	Allowed – reducing the number of parameters would require unnecessary restructuring

Table 15: User Table

2.1.15 MeetingRepository

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed
(8, 7)	Remove this unused import 'java.time.LocalDateTime'	Low	Fixed

Table 16: MeetingRepository Table

2.1.16 MeetingRowMapper

Location	Issue	Severity	Resolution
(0, 0)	Use the "java.time" API for date and time	Info	Fixed

Table 17: MeetingRowMapper Table

2.1.17 UserRepository

Location	Issue	Severity	Resolution
(116, 16)	Update this method so that its implementation is not identical to "resetLoginFailures" on line 103	Medium	Fixed

Table 18: UserRepository Table

2.1.18 UserRowMapper

Location	Issue	Severity	Resolution
(20, 39)	Define a constant instead of duplicating this literal "failedAttempts" 3 times	High	Fixed

Table 19: UserRowMapper Table

2.1.19 LoginSuccessHandler

Location	Issue	Severity	Resolution
(3, 7)	Remove this unused import 'jakarta.servlet.ServletException'.	Low	Fixed

Table 20: LoginSuccessHandler Table

2.1.20 UserService

Location	Issue	Severity	Resolution
(43, 30)	Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method	Info	Fixed
(130, 42)	Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method	Info	Fixed
(15, 17)	Remove this duplicated import	Low	Fixed

Table 21: UserService Table

Overall Assessment: SonarQube identified 59 code quality and security issues during the analysis. 55 of these issues

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

were resolved through code changes. The remaining issues were reviewed and determined to either be acceptable or not require changes at this time. Further improvements can be made in future developments.

2.2 Code Documentation

This section documents the source code of the completed PMSS system. It presents updated UML class diagrams that show how the classes in each package connect to one another, followed by a breakdown of each class in the application, including its attributes, visibility, parameters, and return types.

2.2.1 Class Diagram

The diagrams in this section show the structure of the completed PMSS system using a modified Model-View-Controller (MVC) pattern. This architecture was chosen because it provides strong encapsulation and clearly defines the scope of each package by maintaining a strict separation of concerns across components. The Model is made up of the classes in the model package, and the Controller is made up of the classes in the controller package along with the interfaces they implement. Both the Model and the Controller connect to the supporting config, security, service, and repository packages, which handle authentication, business rules, and data access. The View layer is made up of the Thymeleaf HTML templates rendered by each controller; since these are not Java classes, they are not shown on the class diagrams, but each controller should be understood as connected to the view or views it returns. The diagram below shows how these packages related at the system level, and it is followed by a set of smaller diagrams that isolate the classes belonging to each of PMSS's key functional systems: Login, Registration, User Profile, Meeting Schedule, Complaint, Billing, Room Management, and Reporting.

2.2.1.1 PMSS System Class Diagram

The PMSS system diagram shows a high-level structure of PMSS and how its core components interact across the system. SecurityConfig, in the config package, configures security handlers, authenticates credentials through UserService, logs activity through SecurityLogger, and routes unauthenticated requests to LoginController. Every controller in the controller package relies on UserService to manage active sessions and SecurityLogger to record activity. Data flow is driven by specialized services, repositories, and row mappers, which bridge controllers to instantiate domain entities, such as User, Meeting, and Complaint in the model package. RoomController and ReportController bypass the repository layer to query data directly using Spring's JdbcTemplate. GlobalExceptionHandler is used with the controller package to intercept unhandled exceptions system-wide. It is important to note that this class diagram does not explicitly show the View layer of the MVC model, as those assets are part of the resources/templates package containing solely HTML files for each page of the PMSS, which are not typically modelled in a class diagram.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

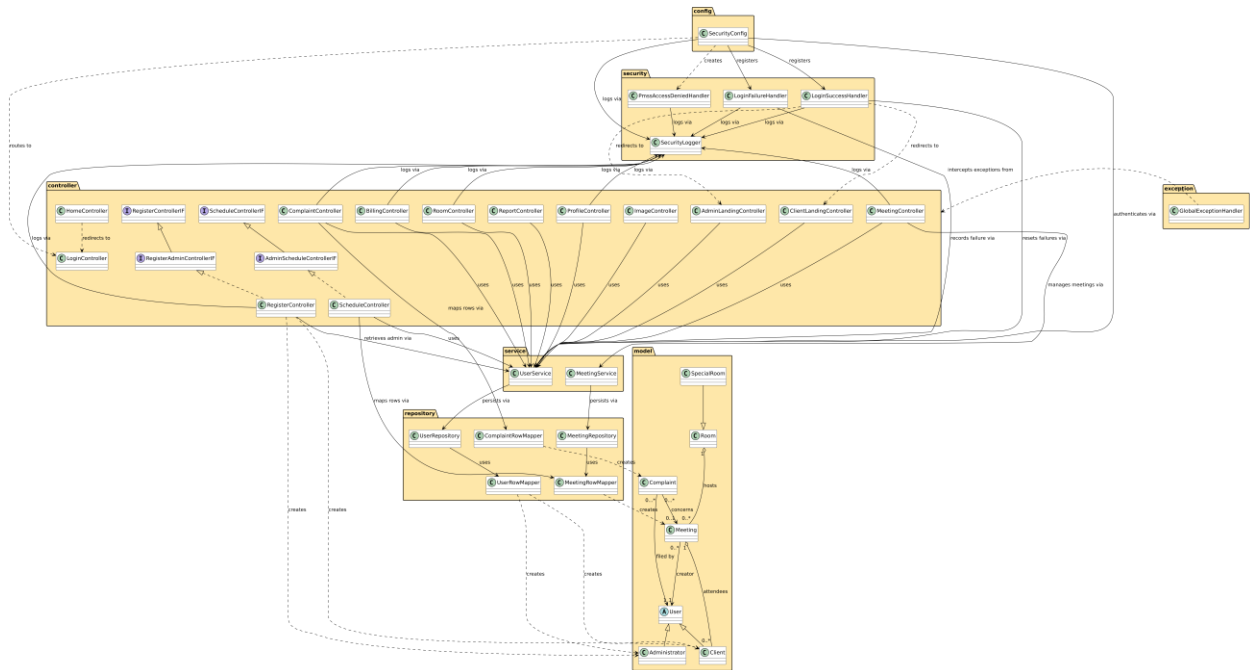


Figure 1: PMSS System Class Diagram

2.2.1.2 Login Subsystem Class Diagram

The Login system diagram shows the classes involved in authenticating a user and directing them to the correct landing page. The SecurityConfig class in the config package defines the login process and registers the LoginSuccessHandler, LoginFailureHandler, and PmssAccessDeniedHandler classes in the security package, which respond to a successful login, a failed login, and an unauthorized access attempt, respectively. Each of these handlers, along with the LoginController that displays the login form, rely on the UserService in the service package to look up the submitted credentials against the User, Client, and Administrator classes in the model package. Once a login succeeds, LoginSuccessHandler redirects the Client or Administrator to the ClientLandingController or AdminLandingController, which retrieve and displays the user's landing page. This is a modified MVC structure because the Controller class and security classes that support them both depend directly on the config package to enforce which requests require authentication before any Model or View is reached.

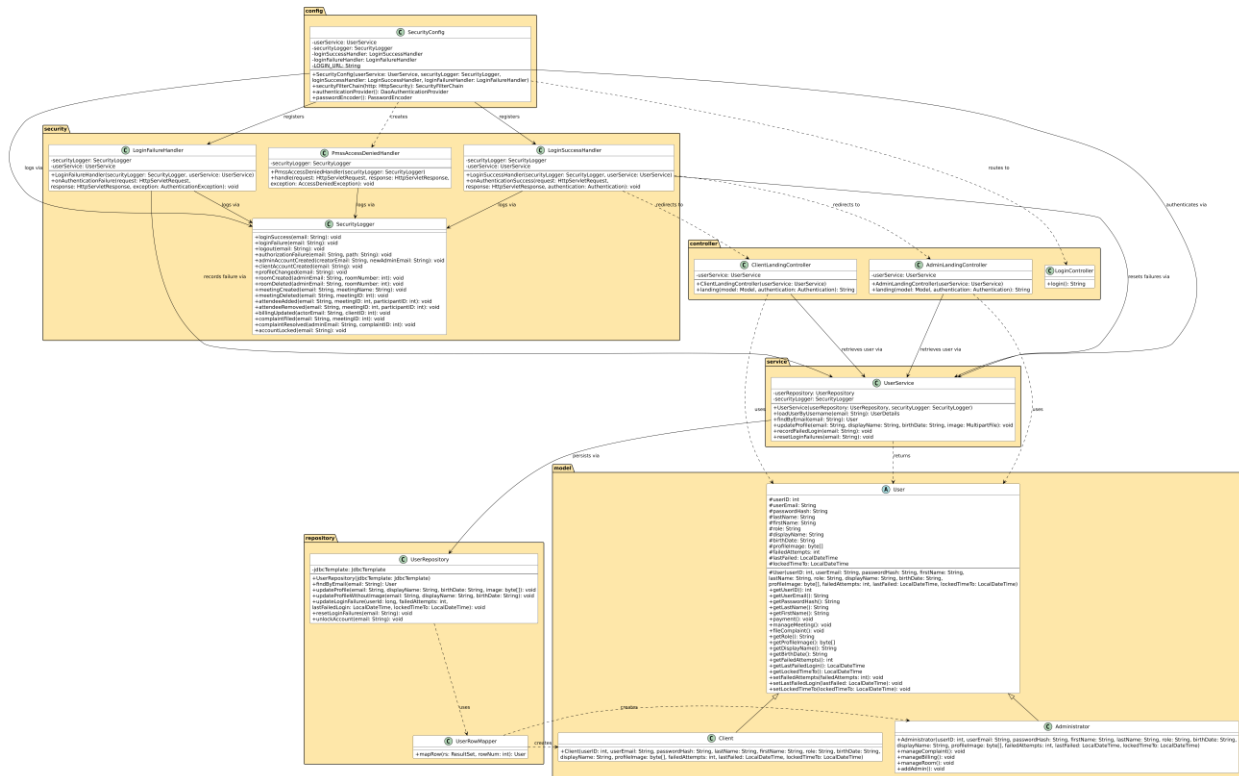


Figure 2: Login Subsystem Class Diagram

2.2.1.3 Registration Subsystem Class Diagram

The Registration system diagram shows the classes used to create new Client and Administrator accounts. The RegisterController class implements the RegisterControllerIF and RegisterAdminControllerIF interfaces, which declare the account-creation methods referenced by the Client and Administrator registration views. RegisterController validates each submitted form, checks the submitted email against existing accounts, and uses Spring's PasswordEncoder to hash a new account's password before creating a new User record in the model package, either as a Client or Adminstrator depending on the form used. It relies on the UserService in the service package only to retrieve the currently authenticated Administrator when displaying the admin registration form, and on the SecurityLogger in the security package to record every account created. The RegisterController performs most of its own validation and persistence rather than delegating to a dedicated Service or Repository class, reflecting the modified MVC structure used throughout PMSS.

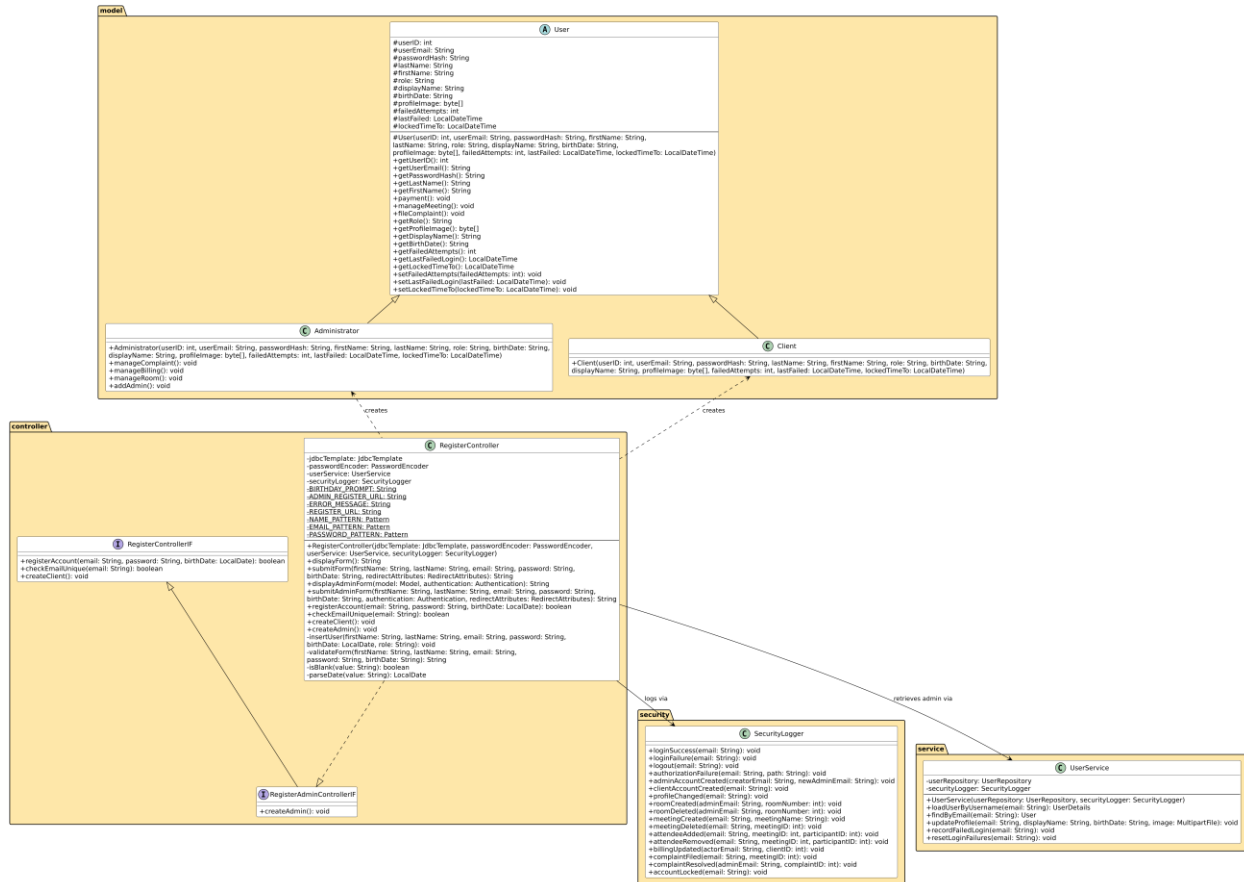


Figure 3: Registration Subsystem Class Diagram

2.2.1.4 User Profile Subsystem Class Diagram

The User Profile system diagram shows the classes used to display and update a user's account information and profile image. The ProfileController class retrieves the authenticated User through the UserService in the service package to display the profile page and edit form, and submits any updated display name, birth date, or profile image to the UserSystem, which persists the change throughout the UserRepository in the repository package. The ImageController class separately retrieves a user's stored profile image through the same UserService to provide the browser with a JPEG file. Both controllers work with the User, Client, and Administrator classes in the model package to represent the account being viewed or edited, and ProfileController records each change through the SecurityLogger in the security package. This subsystem follows the modified MVC structure, with the Controller classes delegating persistence to the Service and Repository packages rather than accessing the database directly.

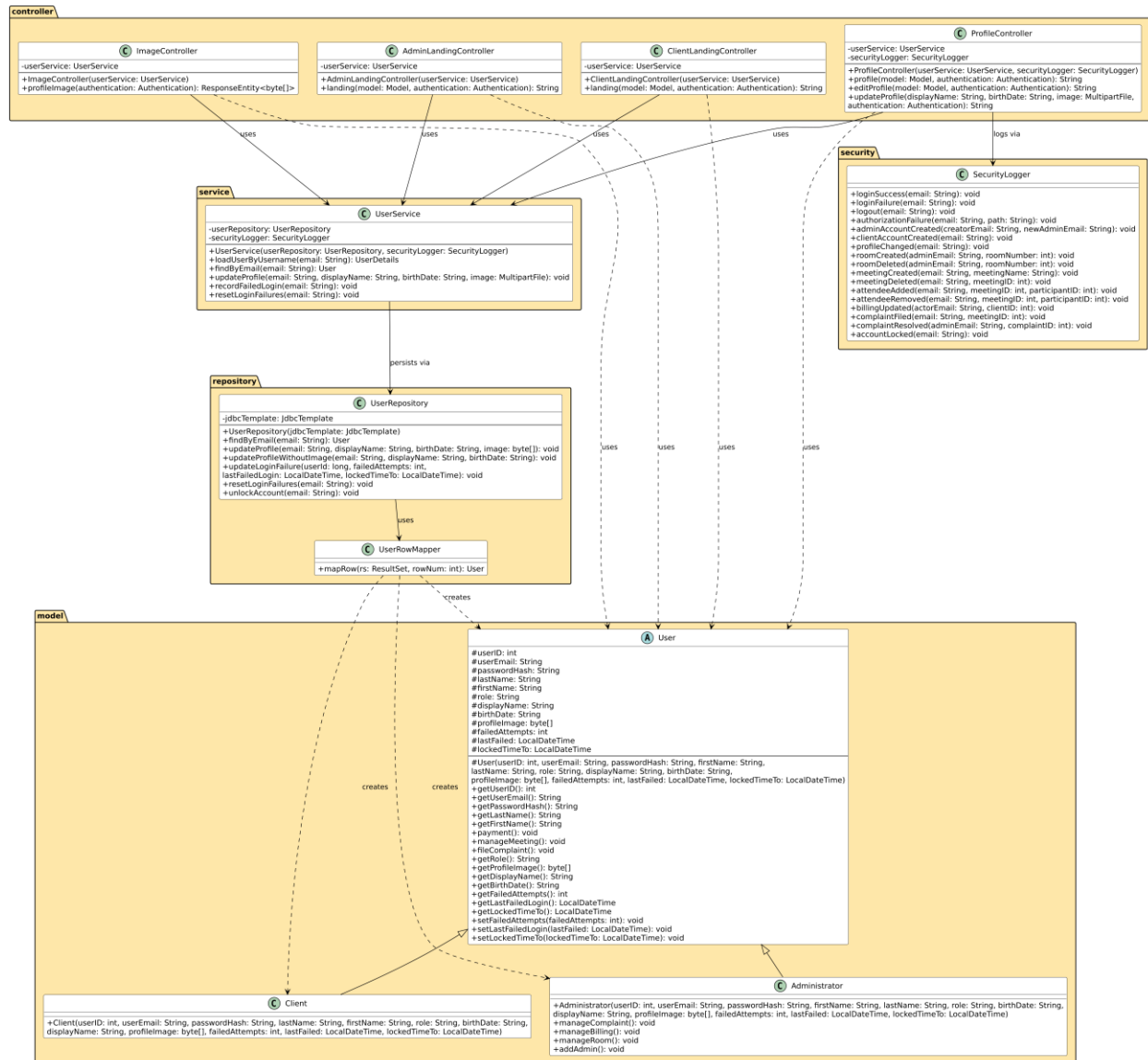


Figure 4: User Profile Subsystem Class Diagram

2.2.1.5 Meeting Schedule Subsystem Class Diagram

The Meeting Schedule system diagram shows the classes used to schedule meetings and view the resulting schedule. The MeetingController class validates and builds new or updated Meeting objects and saves them through the MeetingService in the service package, which confirms room availability before delegating persistence to the MeetingRepository and MeetingRowMapper in the repository package. The ScheduleController class implements the AdminScheduleControllerIF interface, which extends ScheduleControllerIF, and it retrieves and filters existing meetings by day, week, room, person, and time slot for display in the Client or Administrator schedule view. Both controllers also query room and attendee information directly through Spring's JdbcTemplate, reflecting the modified MVC structure in which the Controller may reach past the Service layer to lookup information that is external to the Meeting entity. The Meeting class in the model package identifies the room through a room number shared with the Room and SpecialRoom classes and tracks its attendees as a list of Client objects, and every create, edit, and delete action is recorded through the SecurityLogger in the security package.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

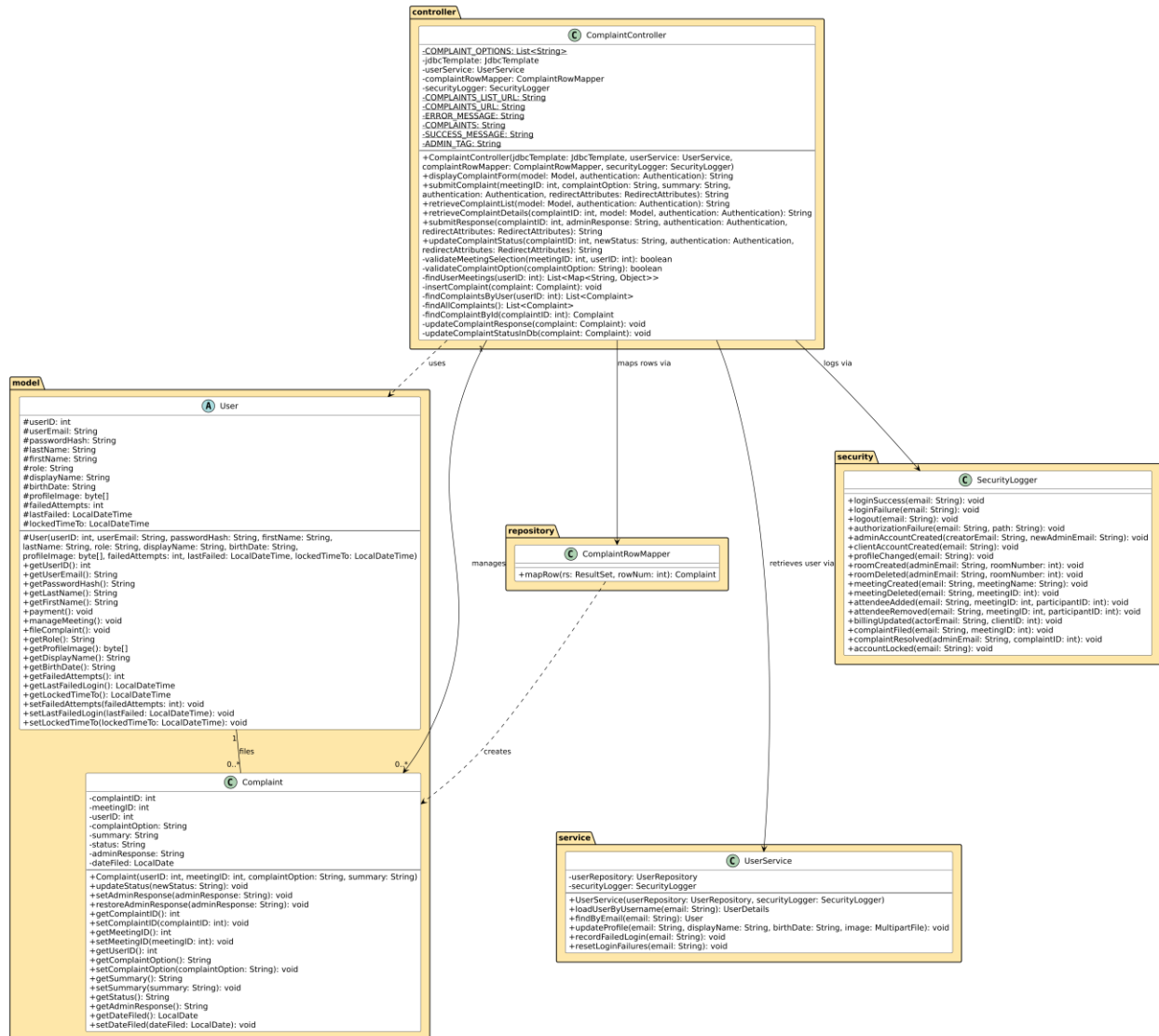


Figure 6: Complaint Subsystem Class Diagram

2.2.1.7 Billing Subsystem Class Diagram

The Billing system diagram shows the classes used to store and manage a Client's payment card. The BillingController class displays a Client's own billing page and lets them save or update their card, and separately gives an Administrator a dedicated view for finding and editing any Client's card on file. It retrieves the authenticated User through the UserService in the service package, but reads and writes the stored card details directly through Spring's JdbcTemplate, masking the card number before it is persisted and never storing the full number or CVV. Every saved or updated card is recorded through the SecurityLogger in the security package. Similar to the Complaint system, this subsystem's Controller performs its own data access rather than delegating to a Repository class.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

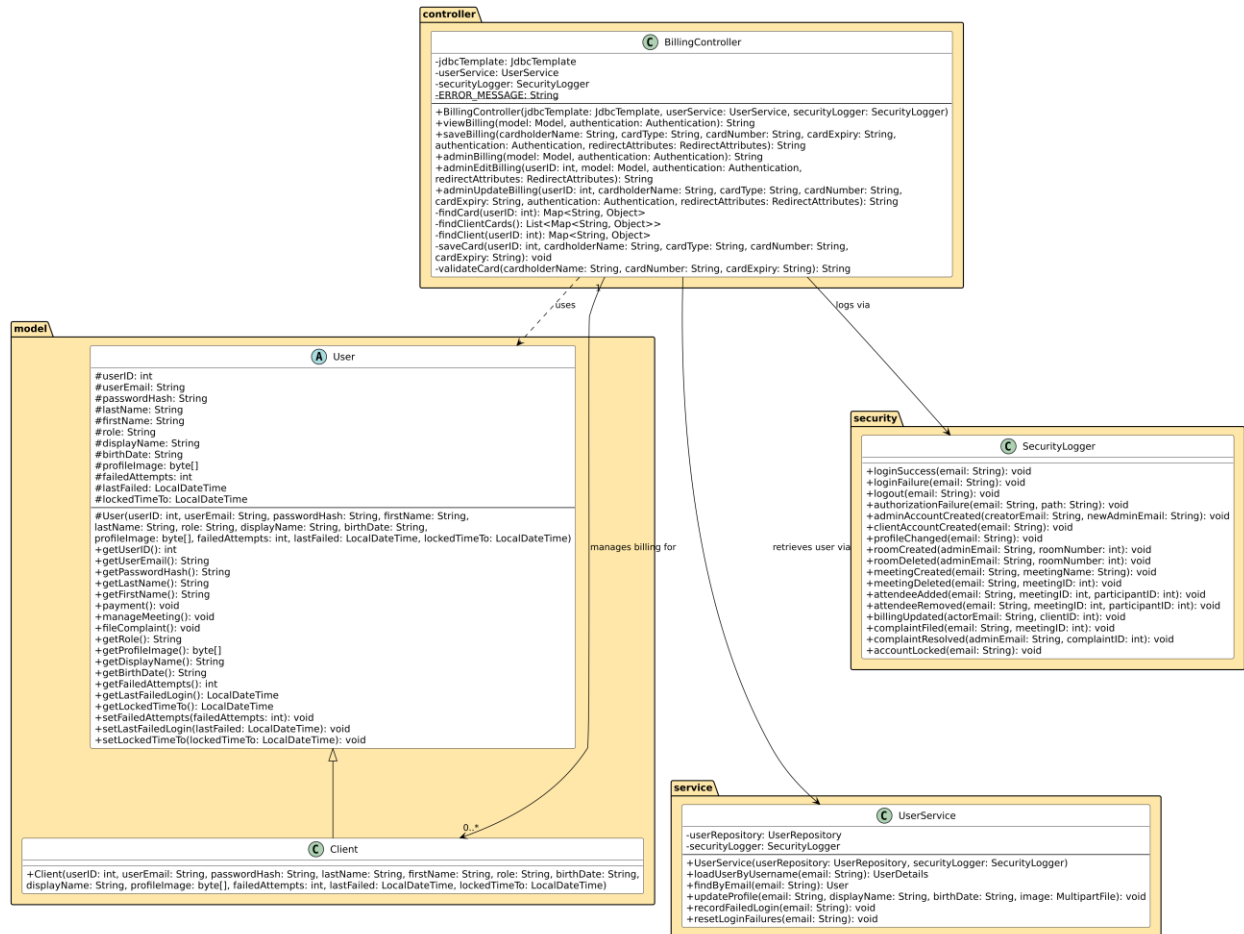


Figure 7: Billing Subsystem Class Diagram

2.2.1.8 Room Management Subsystem Class Diagram

The Room Management system diagram shows the classes used by an Administrator to maintain the set of meeting rooms. The RoomController class lists the existing rooms, adds new Regular or Special rooms, and blocks the deletion of a room that still has scheduled meetings. It retrieves the authenticated Administrator through the UserService in the service package, and reads and writes room records directly through Spring's JdbcTemplate rather than through a dedicated Repository class or the Room and SpecialRoom classes in the model package. Every room added or deleted is recorded through the SecurityLogger in the security package. This subsystem follows the same modified MVC structure as the Complaint and Billing systems, with the Controller managing its own persistence instead of delegating to a Repository.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

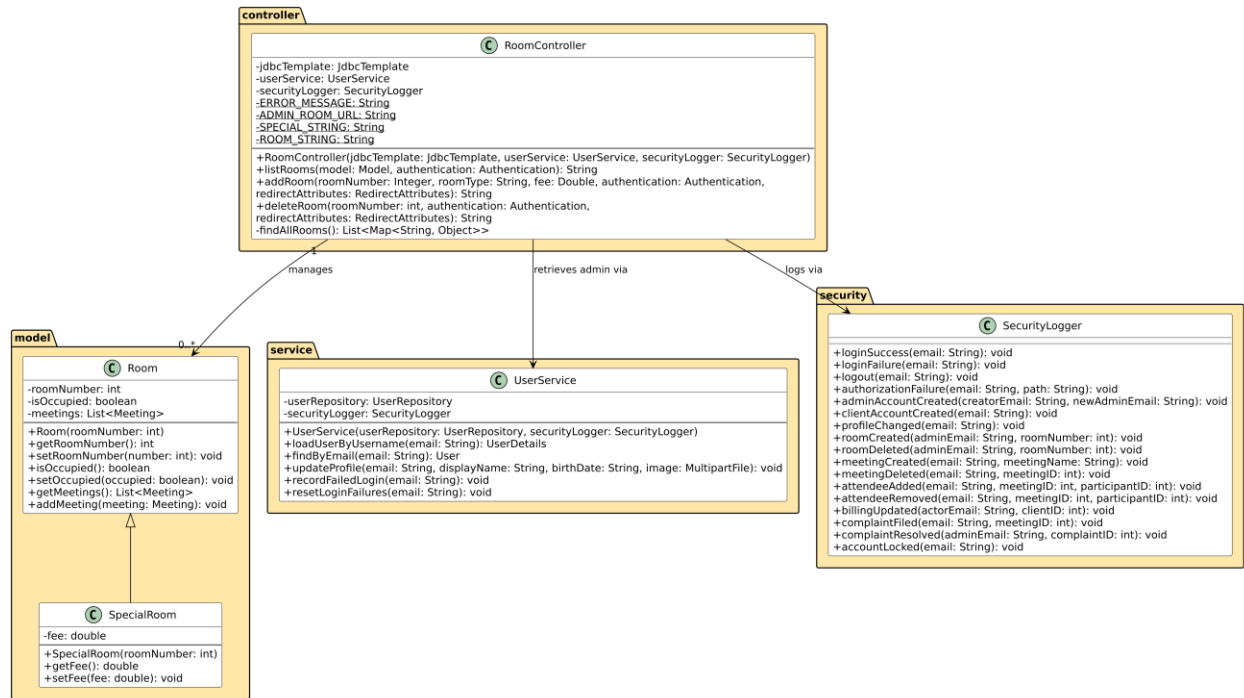


Figure 8: Room Management Subsystem Class Diagram

2.2.1.9 Reporting System Class Diagram

The Reporting system diagram shows the classes used to summarize PMSS's data and export it for an Administrator. The ReportController class retrieves the authenticated Administrator through the UserService in the service package, then queries the Meetings, Rooms, Users, and Complaints tables directly through Spring's JdbcTemplate to build the summary counts, breakdowns, and downloadable CSV files shown on the reports page. Because a report combines data from every other functional area of the system, ReportController does not work with the Meeting, Room, User, or Complaint classes in the model package the way the other controllers do. Instead, it works directly with the raw query results needed to build each count, breakdown, and export. The Controller reads directly from the database across multiple entities rather than through a single Model class or Repository, consistent with the PMSS's modified MVC structure.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

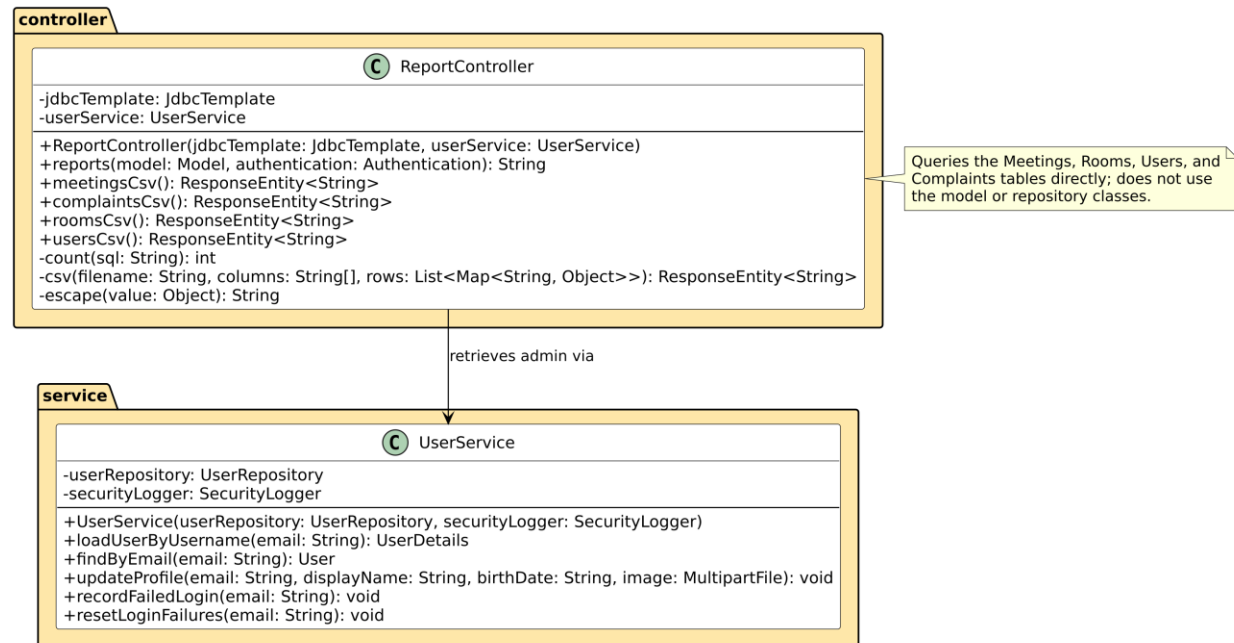


Figure 9: Reporting System Class Diagram

2.2.2 All Classes and Methods

2.2.2.1 Config

The config package defines the application-wide configuration used to secure the PMSS system. The SecurityConfig class establishes the authorization rules that govern which requests require authentication, configures the login and logout process, and registers the password encoder and authentication provider used to validate user credentials. It relies on the UserService in the service package to look up user accounts during authentication, and on the LoginSuccessHandler, LoginFailureHandler, PmssAccessDeniedHandler, and SecurityLogger classes in the security package to route and record the outcome of each authentication attempt. Since every request handled by the controller package passes through the rules it defines, the config package underlies and protects the rest of the system.

2.2.2.1.1 SecurityConfig

The SecurityConfig class is the configuration class for the application's authentication and authorization rules. It shows the main responsibilities of the class, defining which endpoints require authentication, configuring the login and logout workflow, and registering the password encoder and authentication provider used to validate submitted credentials.

Class Name	SecurityConfig		
Attribute	Type	Access	Description
userService	UserService	Private	Reference used to look up accounts during authentication.
securityLogger	SecurityLogger	Private	Reference used to record authentication and authorization events.
loginSuccessHandler	LoginSuccessHandler	Private	Reference used to redirect a user to the correct landing page after a successful login.
loginFailureHandler	LoginFailureHandler	Private	Reference used to redirect a

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

			user back to the login page after a failed login attempt.
LOGIN_URL	String	Private	Reference used for the login endpoint path.
Method	Visibility	Return Type	Description
SecurityConfig(userService: UserService, securityLogger: SecurityLogger, loginSuccessHandler: LoginSuccessHandler, loginFailureHandler: LoginFailureHandler)	Public	N/A	Initializes the configuration with the given user service and security logger.
securityFilterChain(http: HttpSecurity)	Public	SecurityFilterChain	Defines the authorization rule for each request path, configures the login form, logout handling, and access-denied handling, then builds and returns the security filter chain.
authenticationProvider()	Public	DaoAuthenticationProvider	Creates an authentication provider configured with the UserService and password encoder.
passwordEncoder()	Public	PasswordEncoder	Returns the BCryptPasswordEncoder used to hash and verify passwords.

Table 22: SecurityConfig Class Table

2.2.2.2 Controllers

The controller package contains the controller classes that receive incoming requests, coordinate with the Service and Repository classes to retrieve or persist data, and select the view returned to the user. Each controller retrieves the currently authenticated User through the UserService, applies the validation and business rules appropriate to its feature, and forwards Model attributes to the corresponding view. The AdminScheduleControllerIF, RegisterControllerIF, RegisterAdminControllerIF, and ScheduleControllerIF interfaces define the prototypes implemented by the ScheduleController and RegisterController classes, providing the Client and Administrator views role-appropriate scopes.

2.2.2.2.1 AdminLandingController

The AdminLandingController class is the controller for the Administrator landing page. It retrieves the currently authenticated Administrator's user record and forwards it to the landing view.

Class Name	AdminLandingController		
Attribute	Type	Access	Description
userService	UserService	Private	Reference used to retrieve the authenticated user.
Method	Visibility	Return Type	Description
AdminLandingController(UserService: UserService)	Public	N/A	Initializes the controller with the given user service.
landing(model: Model)	Public	String	Retrieves the authenticated administrator's user record, adds it to the

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

authentication: Authentication)			model, and returns the administrator landing view.
---------------------------------	--	--	--

Table 23: AdminLandingController Class Table

2.2.2.2.2 AdminScheduleControllerIF

AdminScheduleControllerIF is the controller interface referenced by the Administrator schedule view. It extends ScheduleControllerIF, inheriting the basic Client retrieval methods and declares additional prototypes used to filter meetings by day, week, room, user, and time slot for Administrator accounts.

Class Name	AdminScheduleControllerIF		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
...	...	...	Inherits getCreatedMeetings(), getParticipatingMeetings(), and getUsersMeetings() from ScheduleControllerIF.
getMeetingsByDay(date: LocalDate)	Public	List<Meeting>	Returns the meetings scheduled on the given date.
getMeetingsByWeek(weekStart: LocalDate)	Public	List<Meeting>	Returns the meetings scheduled during the week containing the given date.
getMeetingsByRoom(room: Room)	Public	List<Meeting>	Returns the meetings scheduled in the given room.
getMeetingsByPerson(user: User)	Public	List<Meeting>	Returns the meetings created by or attended by the given user.
getMeetingsByTimeSlot(startTime: LocalTime, endTime: LocalTime)	Public	List<Meeting>	Returns the meetings that overlap the given time range.

Table 24: AdminScheduleControllerIF Class Table

2.2.2.2.3 BillingController

The BillingController class manages a Client's stored payment method and provides the ability for an Administrator to view and update any Client's billing information. It shows the main responsibilities of the class, including displaying a Client's own billing page, saving, and updating a payment card, and giving Administrators a dedicated view for managing every Client's card on file.

Class Name	BillingController		
Attribute	Type	Access	Description
jdbcTemplate	JdbcTemplate	Private	Reference used to execute billing-related queries.
userService	UserService	Private	Reference used to retrieve the authenticated user.
securityLogger	SecurityLogger	Private	Reference used to record billing updates.
ERROR_MESSAGE	String	Private	Reference used for passing error feedback for unsuccessful form submissions.
Method	Visibility	Return Type	Description
BillingController(jdbcTemplate: JdbcTemplate, userService: UserService,	Public	N/A	Initializes the controller with the given data-access, user-service, and security login dependencies.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

securityLogger: SecurityLogger)			
viewBilling(model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated client and their stored card on file, then displays the billing page.
saveBilling(cardHolderName: String, cardType: String, cardNumber: String, cardExpiry: String, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Validates the submitted card details and saves the authenticated client's card, records the update through the SecurityLogger, and redirects back to the billing page if valid.
adminBilling(model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated administrator and the list of every client's card on file, then displays the admin billing page.
adminEditBilling(userID: int, model: Model, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Searches for the target client and displays the admin billing edit form, pre-filled with the stored cards information. It redirects to the admin billing page with an error if the client does not exist.
adminUpdateBilling(userID: int, cardholderName: String, cardType: String, cardNumber: String, cardExpiry: String)	Public	String	Validates the submitted card details. If valid, it saves the target client's card, records the update through the SecurityLogger, and redirects to the admin billing page.
findCard(userID: int)	Private	Map<String, Object>	Returns the masked card on file for the given user, or null if none is stored.
findClientCards()	Private	Map<String, Object>	Returns every Client account with their stored card details.
findClient(userID: int)	Private	Map<String, Object>	Returns the Client account matching the given ID, or null if no client exists.
saveCard(userID: int, cardholderName: String, cardType: String, cardNumber: String, cardExpiry: String)	Private	Void	Masks the given card number to its last four digits and inserts/updates the stored card record for the given user. The full card number and CVV information are not persisted.
validateCard(cardholderName: String, cardNumber: String, cardExpiry: String)	Private	String	Validates the cardholder name, card number length, and expiry format, returning an error message if any field is invalid or null if the card is valid.

Table 25: BillingController Class Table

2.2.2.2.4 ClientLandingController

The ClientLandingController class is the controller for the Client landing page. It retrieves the currently authenticated Client's user record and forwards it to the landing view.

Class Name	ClientLandingController		
Attribute	Type	Access	Description
userService	UserService	Private	Reference used to retrieve the authenticated user.
Method	Visibility	Return Type	Description

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

ClientLandingController(userService: UserService)	Public	N/A	Initializes the controller with the given user service.
landing(model: Model authentication: Authentication)	Public	String	Retrieves the authenticated client's user record, adds it to the model, and returns the client landing view.

Table 26: ClientLandingController Class Table

2.2.2.2.5 ComplaintController

The ComplaintController class is the primary controller for the Complaint module. It shows the main responsibilities of the class, including displaying the complaining form and complaint list, validating and submitting new complaints, and routing administrator responses and status updates to storage.

Class Name	ComplaintController		
Attribute	Type	Access	Description
COMPLAINT_OPTIONS	List<String>	Private	Fixed list of valid complaint categories a client can select from.
jdbcTemplate	JdbcTemplate	Private	Reference used to execute complaint related queries.
userService	UserService	Private	Reference used to retrieve the authenticated user.
complaintRowMapper	ComplaintRowMapper	Private	Reference used to map query results into Complaint objects.
securityLogger	SecurityLogger	Private	Reference used to record complaint filing and resolution events.
COMPLAINTS_LIST_URL	String	Private	Reference used for the complaint list endpoint path.
COMPLAINTS_URL	String	Private	Reference used for the complaint endpoint path.
ERROR_MESSAGE	String	Private	Reference used for passing error feedback for unsuccessful form submissions.
COMPLAINTS	String	Private	Reference used to identify the complaints view.
SUCCESS_MESSAGE	String	Private	Reference used for passing confirmation feedback for successful form submissions.
ADMIN_TAG	String	Private	Reference used for verifying administrator role authority for access control checks.
Method	Visibility	Return Type	Description
ComplaintController(jdbcTemplate: JdbcTemplate, userService: UserService, complaintRowMapper: ComplaintRowMapper, securityLogger: SecurityLogger)	Public	N/A	Initializes the controller with the given data-access and user-service dependencies.
displayComplaintForm(model: Model, authentication:	Public	String	Retrieves the user's meetings, the available complaint categories, and their filed complaints, then

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Authentication)			displays the complaint form.
submitComplaint(meetingID: int, complaintOption: String, summary: String, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Validates the selected meeting and complaint category, then creates and saves a new complaint.
retrieveComplaintList(model: Model authentication: Authentication)	Public	String	Verifies administrator privileges and displays the full list of filed complaints.
retrieveComplaintDetails(complaintID: int, model: Model, authentication: Authentication)	Public	String	Verifies administrator privileges and displays the complaint matching the given ID.
submitResponse(complaintID: int, adminResponse: String, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Validates and saves the administrator's response, then marks the complaint as resolved.
updateComplaintStatus(complaintID: int, newStatus: String, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Validates and updates the status of the given complaint.
validateMeetingSelection(meetingID: int, userID: int)	Private	Boolean	Returns whether the given meeting ID is blank (no meeting selected) or corresponds to a meeting the given user created/attended.
validateComplaintOption(complaintOption: String)	Private	Boolean	Returns whether the given category is one of the fixed COMPLAINT_OPTIONS.
findUserMeetings(userID: int)	Private	List<Map<String, Object>>	Returns the meetings the given user created or attended to populate the complaint form's meeting selector.
insertComplaint(complaint: Complaint)	Private	Void	Inserts the given complaint into the database.
findComplaintsByUser(userID: int)	Private	List<Complaint>	Returns the complaints filed by the given user, most recently filed first.
findAllComplaints()	Private	List<Complaint>	Returns every complaint in the database, most recently filed first.
findComplaintById(complaintID: int)	Private	Complaint	Returns the complaint matching the given ID, or null if none exists.
updateComplaintResponse(	Private	Void	Saves the given complaint's

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

complaint: Complaint)			administrator response and resolved status to the database.
updateComplaintStatusInDb(complaint: Complaint)	Private	Void	Saves the given complaint's status to the database.

Table 27: ComplaintController Class Table

2.2.2.2.6 HomeController

The HomeController class handles requests made to the application's root path, redirecting visitors to the login page.

Class Name	HomeController		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
home()	Public	String	Redirects the visitor to the login page.

Table 28: HomeController Class Table

2.2.2.2.7 ImageController

The ImageController class is a rest controller responsible for serving as the user's stored profile image. It retrieves the currently authenticated user's image from the UserService and returns them as a JPEG response.

Class Name	ImageController		
Attribute	Type	Access	Description
userService	UserService	Private	Reference used to retrieve the authenticated user's profile image.
Method	Visibility	Return Type	Description
ImageController(userService: UserService)	Public	N/A	Initializes the controller with the given user service.
profileImage(authentication: Authentication)	Public	ResponseEntity<byte[]>	Retrieves the authenticated user's stored profile image and returns it as a JPEG response.

Table 29: ImageController Class Table

2.2.2.2.8 LoginController

The LoginController class is the controller of the login page, displaying the login form to the user.

Class Name	LoginController		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
login()	Public	String	Displays the login form.

Table 30: LoginController Class Table

2.2.2.2.9 MeetingController

The MeetingController class is the primary controller for the Meeting module. It shows the main responsibilities of the class, including displaying the list of meetings, presenting the meeting creation and edit forms, and validating the creating, updating, and deleting meetings and their attendees.

Class Name	MeetingController		
Attribute	Type	Access	Description

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

meetingService	MeetingService	Private	Reference used to create and retrieve meetings.
userService	UserService	Private	Reference used to retrieve the authenticated user.
jdbcTemplate	JdbcTemplate	Private	Reference used to execute room, attendee, and meeting conflict queries.
securityLogger	SecurityLogger	Private	Reference used to record meeting and attendee changes.
MEETING_URL	String	Private	Reference used for the meeting endpoint path.
SUCCESS_MESSAGE	String	Private	Reference used for passing confirmation feedback for successful form submissions.
MEETING_NAME	String	Private	Reference used for the meeting title.
MEETING_PATH	String	Private	Reference used for formatting meeting names in dynamic success messages.
ROOM_NUMBER	String	Private	Reference used for the assigned room identifier.
MEETING_DATE	String	Private	Reference used for the scheduled meeting date.
START_TIME	String	Private	Reference used for the meeting start time.
ERROR_MESSAGE	String	Private	Reference used for passing error feedback for unsuccessful form submissions.
SCHEDULE_URL	String	Private	Reference used for the schedule endpoint path.
END_TIME	String	Private	Reference used for the meeting end time.
EDIT_URL	String	Private	Reference used for the meeting edit endpoint path.
Method	Visibility	Return Type	Description
MeetingController(meetingService: MeetingService, userService: UserService, jdbcTemplate: JdbcTemplate, SecurityLogger: SecurityLogger)	Public	N/A	Initializes the controller with the given meeting-service and user-service dependencies.
displayMeetings()	Public	String	Redirects to the Schedule page to view the consolidated meeting list.
showCreateMeetingPage(model: Model, authentication: Authentication)	Public	String	Initializes a blank meeting and displays the meeting creation form.
createMeeting(meetingName: String, meetingDate: String, startTime: String, endTime: String, roomNumber: Integer, confirmed: boolean, model: Model,	Public	String	Builds a new meeting from the submitted form fields, assigns the authenticated user as its creator, sets its status to Scheduled, and saves it through the MeetingService.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

authentication: Authentication, redirectAttributes: RedirectAttributes)			
showEditMeetingPage(id: int, model: Model, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Retrieves the meeting matching the given ID, If the authenticated user is the meeting creator, it populates and displays the meeting edit form.
updateMeeting(id: int, meetingName: string, meetingDate: String, startTime: String, endTime: String, roomNumber: Integer, model: Model, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Verifies the authenticated user is the meeting's creator, validates the submitted details and room/creator availability, then saves the changes through the MeetingService.
deleteMeeting(id: int, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Verifies the authenticated user is the meeting's creator, removes its attendees and the meeting, and records the deletion through the SecurityLogger.
addParticipant(id: int, participantID: Integer, authentication: Authentication, redirectAttribute: RedirectAttributes)	Public	String	Verifies the authenticated user is the meeting's creator and that the selected participant is available during the meeting's time, then adds the participant to the attendee list and records the addition through the SecurityLogger.
removeParticipant (id: int, participantID: Integer, authentication: Authentication, redirectAttribute: RedirectAttributes)			Verifies the authenticated user is the meeting's creator, removes the given participant from the attendee list, and records the removal through the SecurityLogger.
addFormOptions(model: Model)	Private	Void	Adds the list of available rooms and the fixed set of business-hour time slots to the model for the create and edit meeting forms.
validateMeeting(name: String, date: String, start: String, end: String, roomNumber: Integer)	Private	String	Validates that the meeting name, date, room, and time fields are present, fall within business hours, and that the end time is after the start time, returning an error message or null if the meeting is valid.
roomIsAvailable(roomNumber: int, date: String, startFull: String, endFull: String, excludeMeetingID: int)	Private	Boolean	Returns whether the given room has no other meeting overlapping the given date and time range.
findRoom(roomNumber: int)	Private	Map<String, Object>	Returns the room matching the given number, or null if it does not exist.
requireCreator(meeting: Meeting, user: User,	Private	String	Returns a redirection to the Schedule page with an error message if the

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

redirectAttributes: RedirectAttributes)			given meeting does not exist or the given user is not its creator, or null if the user may proceed.
populateEditModel(model: Model, meeting: Meeting)	Private	Void	Adds the given meeting's details, form options, attendees, and addable clients to the model for the edit meeting form.
findAttendees(meetingId: int)	Private	List<Map<String, Object>>	Returns the clients attending the given meeting.
findAddableClients(meetingID: int, creatorID: int)	Private	List<Map<String, Object>>	Returns the users who are not the meeting's creator and are not already attending it, to populate the add-participant selector.
clientIsAvailable(userID: int, date: String, startFull: String, endFull: String, excludeMeetingID: int)	Private	Boolean	Returns whether the given user has no created or attended meetings, overlapping the given data and time range.

Table 31: MeetingController Class Table

2.2.2.2.10 ProfileController

The ProfileController class is the primary controller for the client's profile. It shows the main responsibilities of the ProfileController, including displaying the profile, presenting the profile edit form, and updating the profile through the UserService.

Class Name	ProfileController		
Attribute	Type	Access	Description
userService	UserService	Private	Preference used to retrieve and update the authenticated user's profile.
securityLogger	SecurityLogger	Private	Reference used to record profile changes.
Method	Visibility	Return Type	Description
ProfileController(userService: UserService, securityLogger: SecurityLogger)	Public	N/A	Initializes the controller with the given user-service and security-login dependencies.
profile(model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated user and displays their profile.
editProfile(model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated user and displays the profile edit form.
updateProfile(displayName: String, birthDate: String, image: MultipartFile, authentication: Authentication)	Public	String	Submits the updated display name, birth date, and optional profile image to the UserService, records the change through the SecurityLogger, and returns to the profile.

Table 32: ProfileController Class Table

2.2.2.2.11 RegisterAdminControllerIF

RegisterAdminControllerIF is the controller interface that inherits from RegisterControllerIF and is referenced by the Administrator registration view. It adds the createAdmin prototype implemented by its concrete subclasses.

Class Name	RegisterAdminControllerIF
------------	---------------------------

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
...	...	...	Inherits registerAccount(), checksEmailUnique(), and createClient() from RegisterControllerIF.
createAdmin()	Public	Void	Creates a new Administrator account from the registration details.

Table 33: RegisterAdminControllerIF Class Table

2.2.2.2.12 RegisterController

The RegisterController class is the controller implementation for the Register module. It implements RegisterAdminControllerIF, which extends RegisterControllerIF, so it provides the account-creation method used to register both Client and Administrator accounts, validating submitted forms, checking for a unique email, and creating the corresponding account.

Class Name	RegisterController		
Attribute	Type	Access	Description
jdbcTemplate	JdbcTemplate	Private	Reference used to check email uniqueness and insert new accounts.
passwordEncoder	PasswordEncoder	Private	Reference used to hash a new account's password before it is stored.
userService	UserService	Private	Reference used to retrieve the authenticated administrator for the admin registration form.
securityLogger	SecurityLogger	Private	Reference used to record Client and Administrator account creation.
BIRTHDAY_PROMPT	String	Private	Reference containing birth date prompt message.
ADMIN_REGISTER_URL	String	Private	Reference used for the register administrator endpoint path.
ERROR_MESSAGE	String	Private	Reference used for passing error feedback for unsuccessful form submissions.
REGISTER_URL	String	Private	Reference used for the register endpoint path.
NAME_PATTERN	Pattern	Private	Expression used to validate first and last names.
EMAIL_PATTERN	Pattern	Private	Expression used to validate that a submitted email ends with @pennstatesoft.com.
PASSWORD_PATTERN	Pattern	Private	Expression used to require at least one digit and one special character in a submitted password.
Method	Visibility	Return Type	Description
RegisterController(jdbcTemplate: JdbcTemplate, passwordEncoder: PasswordEncoder, userService: UserService, securityLogger: SecurityLogger)	Public	N/A	Initializes the controller with the given data-access, password-encoding, and user-service dependencies.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

SecurityLogger)			
displayForm()	Public	String	Displays the Client registration form
submitForm(firstName: String, lastName: String, email: String, password: String, birthDate: String, redirectAttributes: RedirectAttributes)	Public	String	Validates the submitted fields, stages the pending account, and registers a new Client account.
displayAdminForm(model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated administrator and displays the Administrator registration form.
submitAdminForm(firstName: String, lastName: String, email: String, password: String, birthDate: String, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Validates the submitted fields, stages the pending account, checks that the email is unique, and creates a new Administrator account.
registerAccount(email: String, password: String, birthDate: LocalDate)	Public	String	Checks that the given email is unique. If so, it creates a new Client account with blank name fields from the given credentials and birth date. It returns whether the account was created.
checkEmailUnique(email: String)	Public	Boolean	Checks the database for the given email address and returns whether it is still available and not already claimed by an existing account.
createClient()	Public	Void	Throws an UnsupportedOperationException, since Client creation is performed directly by insertUser() from submitForm().
createAdmin()	Public	Void	Throws an UnsupportedOperationException, since Administrator account creation is performed directly by insertUser() from submitAdminForm().
insertUser(firstName: String, lastName: String, email: String, password: String, birthDate: LocalDate, role: String)	Private	Void	Hashes the given password and inserts a new account of the given role with the given name, email, and birth date.
validateForm(firstName: String, lastName: String, email: String, password: String, birthDate: String)	Private	String	Validates the length and format of the submitted name, email, password, and birth date fields, returning an error message describing the first invalid field or null if the form is valid.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

isBlank(value: String)	Private	Boolean	Returns whether the given value is null or contains only whitespace.
parseDate(Value: String)	Private	LocalDate	Parses the given birth date string into a Date at the start of that day in the system's default time zone.

Table 34: RegisterController Class Table

2.2.2.2.13 RegisterControllerIF

RegisterControllerIF is the controller interface implemented by RegisterController and referenced by the Client registration view. It declares the account-creation prototypes used to register a new Client account.

Class Name	RegisterControllerIF		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
registerAccount(email: String, password: String, birthDate: LocalDate)	Public	Boolean	Driver method for creating a new account, implemented by concrete classes to stage and create a Client account.
checkEmailUnique(email: String)	Public	Boolean	Checks the database for if the email address has already been claimed.
createClient()	Public	Void	Creates a new Client account.

Table 35: RegisterControllerIF Class Table

2.2.2.2.14 ReportController

The ReportController class is the primary controller for Administrator reporting. It shows the main responsibilities of the class, including displaying summary statistics and breakdowns of meetings, rooms, users, and complaints, and exporting each of those record types as a downloadable CSV file.

Class Name	ReportController		
Attribute	Type	Access	Description
jdbcTemplate	JdbcTemplate	Private	Reference used to execute the queries backing each report and export.
userService	UserService	Private	Reference used to retrieve the authenticated administrator.
Method	Visibility	Return Type	Description
ReportController(jdbcTemplate: JdbcTemplate, userService: UserService)	Public	N/A	Initializes the controller with the given data-access and user-service dependencies.
reports(model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated administrator and the set of summary counts and breakdowns: totals (for meetings, rooms, special rooms, clients, administrators, and complaints), meetings by room and creator, and complaints by status. It displas the reports page.
meetingsCsv()	Public	ResponseEntity<String>	Returns a downloadable

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

			meetings.csv containing every meeting with its creator's name.
complaintsCsv()	Public	ResponseEntity<String>	Returns a downloadable complaints.csv file containing every complaint with the client's name.
roomsCsv()	Public	ResponseEntity<String>	Returns a downloadable rooms.csv file containing every room with its meeting count.
usersCsv()	Public	ResponseEntity<String>	Returns every user's non-sensitive account details (excludes password hash and profile image) as a downloadable users.csv file.
count(sql: String)	Private	Int	Executes the given count query and returns the result, or 0 if no row is returned.
csv(filename: String, columns: String[], rows: List<Map<String, Object>>)	Private	ResponseEntity<String>	Builds a CSV document from the given column headers and rows and returns it as a file-download response with the given filename.
escape(value: Object)	Private	String	Returns the given value as a CSV-safe string, escaping if it contains a comma, quotation mark, or newline.

Table 36: ReportController Class Table

2.2.2.2.15 RoomController

The RoomController class is the primary controller for Administrator room management. It shows the main responsibilities of the class, including listing the meeting rooms, adding new Regular or Special rooms, and deleting rooms that have no scheduled meetings.

Class Name	RoomController		
Attribute	Type	Access	Description
jdbcTemplate	JdbcTemplate	Private	Reference used to execute room-related queries.
userService	UserService	Private	Reference used to retrieve the authenticated administrator.
securityLogger	SecurityLogger	Private	Reference used to record room creation and deletion.
ERROR_MESSAGE	String	Private	Reference used for passing error feedback for unsuccessful form submissions.
ADMIN_ROOM_URL	String	Private	Reference used for the admin room view endpoint path.
SPECIAL_STRING	String	Private	Reference used for verifying special room classification.
ROOM_STRING	String	Private	Reference used for formatting room names in dynamic messages.
Method	Visibility	Return Type	Description
RoomController(jdbcTemplate: JdbcTemplate, userService: UserService, securityLogger: SecurityLogger)	Public	N/A	Initializes the controller with the given data-access, user-service, and security-logging dependencies.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

listRooms(model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated administrator and all rooms, displaying the room management page.
addRoom(roomNumber: Integer, roomType: String, fee: Double, authentication: Authentication, redirectAttributes: RedirectAttributes)	Public	String	Validates that the room number is valid and not already in use, then inserts a Regular or Special room and records the creation through the SecurityLogger.
deleteRoom(roomNumber: int, authentication: Authentication, redirectAttribute: RedirectAttribute)	Public	String	Blocks deletion of a room that has scheduled meetings, otherwise deletes the room and records the deletion through the SecurityLogger.
findAllRooms()	Private	List<Map<String, Object>>	Returns all rooms, ordered by room number.

Table 37: RoomController Class Table

2.2.2.2.16 ScheduleController

The ScheduleController class is the primary controller for the Schedule module. It implements AdminScheduleControllerIF, and shows the main responsibilities of the class, including displaying the Client and Administrator schedule views, retrieving meetings created by or attending by a given user, and filtering meetings by day, week, room, person, and time slot for Administrators.

Class Name	ScheduleController		
Attribute	Type	Access	Description
jdbcTemplate	JdbcTemplate	Private	Reference used to execute schedule-related queries.
userService	UserService	Private	Reference used to retrieve the authenticated user.
meetingRowMapper	MeetingRowMapper	Private	Reference used to map query results into Meeting objects.
meetings	List<Meeting>	Private	Cached list of the meetings most recently retrieved for display.
Method	Visibility	Return Type	Description
ScheduleController(jdbcTemplate: JdbcTemplate, userService: UserService, meetingRowMapper: MeetingRowMapper)	Public	N/A	Initializes the controller with the given data-access, user-service, and row-mapper dependencies.
displaySchedule(view: String, filter: String, day: String, weekStart: String, roomNumber: Integer, personID: Integer, timeSlot: String, model: Model, authentication: Authentication)	Public	String	Retrieves the authenticated user and renders either the Administrator or Client schedule view, applying any requested filter.
renderClientDashboard(model: Model, user: User, view: String)	Private	String	Selects the Client's meetings (all, created only, or participating only) based on the given view, and adds them to the model for the schedule view.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

renderAdminDashboard(model: Model, user: User, filter: String, day: String, weekStart: String, roomNumber: Integer, personID: Integer, timeSlot: String)	Private	String	Selects the meetings matching the given filter (by day, week, room, person, time slot, creator, attendee, or every meeting by default) and adds them with the room, person, and time-slot options to the model for the Administrator schedule view.
getCreatedMeetings(user: User)	Public	List<Meeting>	Returns the meetings created by the given user.
getParticipatingMeetings(user: User)	Public	List<Meeting>	Returns the meetings the given user is attending but did not create.
getUsersMeetings(user: User)	Public	List<Meeting>	Returns the meetings the given user has created or is attending.
getMeetingsByDay(date: LocalDate)	Public	List<Meeting>	Returns the meeting scheduled on the given date.
getMeetingsByWeek(weekStart: LocalDate)	Public	List<Meeting>	Returns the meetings scheduled between the Monday and Sunday of the week containing the given date.
getMeetingsByRoom(room: Room)	Public	List<Meeting>	Returns the meetings scheduled in the given room.
getMeetingsByPerson(user: User)	Public	List<Meeting>	Returns the meetings created by or attended by the given user.
getMeetingsByTimeSlot(startTime: LocalTime, endTime: LocalTime)	Public	List<Meeting>	Returns the meetings that overlap the given time range.
findAllMeetings()	Private	List<Meeting>	Returns all meetings, ordered by date and start time.
findUserById(userID: int)	Private	User	Returns the user matching the given ID, or null if no such user exists.
findAllRooms()	Private	List<Map<String, Object>>	Returns all rooms for the room filter selector, ordered by room number.
findAllUsers()	Private	List<Map<String, Object>>	Returns all users for the person filter selector, ordered by name.
businessHourSlots()	Private	Map<String, String>	Returns an ordered map of each business-hour slot's start time to its "start-end" display label, used to populate the time-slot filter
normalizeTime(value: String)	Private	LocalTime	Parses the given time string into a Time, appending seconds if only hours and minutes were given.
plusOneHour(time: LocalTime)	Private	LocalTime	Returns the given time incremented by one hour.

Table 38: ScheduleController Class Table

2.2.2.2.17 ScheduleControllerIF

ScheduleControllerIF is the controller interface referenced by the Client schedule view. It limits Clients to only the meeting-retrieval methods they are permitted to call.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Class Name	ScheduleControllerIF		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
getCreatedMeetings(user: User)	Public	List<Meeting>	Returns the meetings created by the given user.
getParticipatingMeetings(user: User)	Public	List<Meeting>	Returns the meetings the given user is attending but did not create.
getUsersMeetings(user: User)	Public	List<Meeting>	Returns the meetings the given user has created or is attending.

Table 39: ScheduleControllerIF Class Table

2.2.2.3 Exceptions

The exception package contains the application-wide exception handling for PMSS. The GlobalExceptionHandler class intercepts any exception that escapes a controller's handler method, logs it, and renders an error page in place of a stack trace or a whitelabel error page, distinguishing between a missing database record, a malformed request, and an unexpected server error.

2.2.2.3.1 GlobalExceptionHandler

The GlobalExceptionHandler class is the global exception handler for every controller in PMSS. It shows the main responsibilities of the class, including catching a missing database record, a malformed request, and any other unhandled exception, logging each occurrence, and rendering the shared error page with a status code and message to the user.

Class Name	GlobalExceptionHandler		
Attribute	Type	Access	Description
log	Logger	Private	Logger used to record exceptions caught by the handler.
ERROR_STRING	String	Private	Reference used for formatting errors in dynamic exception messages.
MESSAGE_STRING	String	Private	Reference used for formatting missing component dynamic exception messages.
STATUS_STRING	String	Private	Reference used for formatting invalid input dynamic exception messages.
Method	Visibility	Return Type	Description
handleNotFound(ex: EmptyResultDataAccessException, model: Model)	Public	String	Logs the missing-record exception and displays the error page with a 404 status and a message indicating the request item could not be found.
handleBadRequest(ex: Exception, model: Model)	Public	String	Logs the malformed-request exception and displays the error page with a 400 status and a message indicating the request was not valid.
handleUnexpected(ex: Exception, model: Model)	Public	String	Logs any other unhandled exception and displays the error page with a 500 status and a generic message.

Table 40: GlobalExceptionHandler Class Table

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

2.2.2.4 Models

The model package contains the entity classes that represent the core data of the PMSS system, including its Users, Meetings, Rooms, and Complaints. The abstract User class defines the attributes and account behavior shared by every account, while the Client and Administrator classes extend it to represent the systems' two account types. The Meeting class references the Client class through its list of attendees, and the Room and SpecialRoom classes are referenced by Meeting through its room number. The Complaint class references the Meeting and User it was filed against and under. These classes are constructed and populated by the RowMapper classes in the repository package, ready and written to the database by the repository package, and are passed to views by the controller package

2.2.2.4.1 Administrator

The Administrator class represents the model entity for the Administrator user. It extends the abstract User class and shows the main responsibilities of the class, including managing complaints, bills, rooms, and creating new administrator accounts.

Class Name	Administrator		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
Administrator(userID: int, userEmail: String, passwordHash: String, firstName: String, lastName: String, role: String, birthDate: String, displayName: String, profileImage: byte[], failedAttempts: int, lastFailed: LocalDateTime, lockedTimeTo: LocalDateTime)	Public	N/A	Initializes a new administrator by passing the given account details to the User constructor.
manageComplaint()	Public	Void	Reviews and updates complaint details.
manageBilling()	Public	Void	Reviews and updates billing records.
manageRoom()	Public	Void	Adds or updates room information.
addAdmin()	Public	Void	Creates an administrator account

Table 41: Administrator Class Table

2.2.2.4.2 Client

The Client class represents the model entity for the Client user. It extends the abstract User class and inherits its attributes and account-management behavior.

Class Name	Client		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
Client(userID: int, userEmail: String, passwordHash: String, lastName: String, firstName: String, role: String, birthDate: String, displayName: String,	Public	N/A	Initializes a new client by passing the given account details to the User constructor.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

profileImage: byte[], failedAttempts: int, lastFailed: LocalDateTime, lockedTimeTo: LocalDateTime)			
---	--	--	--

Table 42: Client Class Table

2.2.2.4.3 Complaint

The Complaint class serves as the core model entity representing a complaint filed by a client. It encapsulates private complaint details while providing public methods to modify status states and set an administrator response.

Class Name	Complaint		
Attribute	Type	Access	Description
complaintID	Int	Private	Identifier unique to the complaint.
meetingID	Int	Private	Identifier of the meeting the complaint concerns if applicable.
userID	Int	Private	Identifier of the user who filed the complaint.
complaintOption	String	Private	Complaint category selected by the client.
summary	String	Private	Text description of the complaint entered by the client.
status	String	Private	Current status of the complaint.
adminResponse	String	Private	Administrator response to the complaint.
dateFiled	LocalDate	Private	Date the complaint was submitted.
Method	Visibility	Return Type	Description
Complaint(userID: int, meetingID: int, complaintOption: String, summary: String)	Public	N/A	Initializes a new complaint with the given user, meeting, option, and summary setting its status to Pending and recording the filing date.
updateStatus(newStatus: String)	Public	Void	Updates the complaint's status.
setAdminResponse(adminResponse: String)	Public	Void	Saves the administrator's response and makes the complaint as Resolved.
restoreAdminResponse(adminResponse: String)	Public	Void	Restores the previously saved administrator response without changing the complaint's status.
getComplaintID()	Public	Int	Returns the complaint's ID.
setComplaintID(complaintID: int)	Public	Void	Sets the complaint's ID.
getMeetingID()	Public	Int	Returns the ID of the meeting the complaint concerns.
setMeetingID(meetingID: int)	Public	Void	Sets the ID of the meeting the complaint concerns.
getUserID()	Public	Int	Returns the ID of the user who filed the complaint.
getComplaintOption()	Public	String	Returns the complaint's category.
setComplaintOption(complaintOption: String)	Public	Void	Sets the complaint's category.
getSummary()	Public	String	Returns the complaint's summary.
setSummary(summary: String)	Public	Void	Sets the complaint's summary.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

getStatus()	Public	String	Returns the complaints status.
getAdminResponse()	Public	String	Returns the administrator's response to the complaint.
getDateFiled()	Public	LocalDate	Returns the date the complaint was filed.
setDateFiled(dateFiled: LocalDate)	Public	Void	Sets the date the complaint was filed.

Table 43: Complaint Class Table

2.2.2.4.4 Meeting

The Meeting class represents a meeting created by a client and scheduled in a meeting room. It encapsulates the meeting's scheduling details and lists the attendees while providing the public accessors and mutators used by the controller package.

Class Name	Meeting		
Attribute	Type	Access	Description
meetingID	Int	Private	Unique identifier assigned to the meeting.
meetingDate	LocalDate	Private	Date that the meeting is scheduled on.
startDate	LocalTime	Private	Starting time of the meeting.
endDate	LocalTime	Private	Ending time of the meeting.
creatorID	Int	Private	Identifier of the client who created the meeting.
roomNumber	Int	Private	Number of the room reserved for the meeting.
attendees	List<Client>	Private	List of clients attending the meeting.
status	String	Private	Meeting status.
roomId	Int	Private	Numeric room identifier used by MeetingService when checking a room's availability through MeetingRepository.
meetingName	String	Private	Name or title of the meeting.
Method	Visibility	Return Type	Description
Meeting(meetingID: int, meetingName: String, meetingDate: LocalDate, roomNumber: int)	Public	N/A	Initializes a new meeting with the given identifier, name, date, and room, with an empty attendee list.
getMeetingID()	Public	Int	Returns the meeting's unique identifier.
getMeetingName()	Public	String	Returns the meeting's name.
setMeetingName(meetingName: String)	Public	Void	Sets the meeting's name.
getMeetingDate()	Public	LocalDate	Returns the meeting's scheduled time.
setMeetingDate(meetingDate: LocalDate)	Public	LocalDate	Sets the meeting scheduled time.
getStartTime()	Public	LocalTime	Returns the meeting's start time.
setStartTime(startTime: localTime)	Public	Void	Sets the meeting's start time.
getEndTime()	Public	LocalTime	Returns the meeting's ending time.
setEndTime(endTime: LocalTime)	Public	Void	Sets the meeting's ending time.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

getCreatorID()	Public	Int	Returns the identifier of the meeting's creator.
setCreatorID(creatorID: int)	Public	Void	Sets the identifier of the meeting's creator.
getRoomNumber()	Public	Int	Returns the given room number to the meeting.
setRoomNumber(roomNumber: int)	Public	Void	Assigns the given room number to the meeting.
getAttendees()	Public	List<Client>	Returns the list of clients attending the meeting.
addAttendee(client: Client)	Public	Boolean	Adds the given client to the attendee list if they are not already present.
removeAttendee(userID: int)	Public	Boolean	Removes the client with the given ID from the attendee list.
getStatus()	Public	String	Returns the meeting's current status.
setStatus(status: String)	Public	Void	Sets the meeting's status.
getRoomId()	Public	Int	Returns the meeting's stored room identifier.

Table 44: Meeting Class Table

2.2.2.4.5 Room

The Room class represents the model entity for a meeting room available for scheduling. It tracks whether the room is occupied and maintains the list of meetings scheduled within it.

Class Name	Room		
Attribute	Type	Access	Description
roomNumber	Int	Private	Number identifying the room.
isOccupied	Boolean	Private	Whether the room is currently occupied.
meetings	List<Meeting>	Private	List of meetings scheduled in the room.
Method	Visibility	Return Type	Description
Room(roomNumber: int)	Public	N/A	Initializes a new room with the given number and an empty meeting list.
getRoomNumber()	Public	Int	Returns the room's number.
setRoomNumber(number: int)	Public	Void	Sets the room's number.
isOccupied()	Public	Boolean	Returns whether the room is occupied.
setOccupied(occupied: boolean)	Public	Void	Sets whether the room is occupied.
getMeetings()	Public	List<Meeting>	Returns the list of meetings scheduled in the room.
addMeeting(meeting: Meeting)	Public	Void	Adds the given meeting to the room's meeting list.

Table 45: Room Class Table

2.2.2.4.6 SpecialRoom

The SpecialRoom class extends the Room class to represent a premium meeting room that carries an associated reservation fee. The main responsibility of this class is to store and provide access to that fee.

Class Name	SpecialRoom		
Attribute	Type	Access	Description

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

fee	Double	Private	Fee charged to reserve the special room.
Method	Visibility	Return Type	Description
SpecialRoom(roomNumber: int)	Public	N/A	Initializes a new special room with the given number and a default fee.
getFee()	Public	Double	Returns the room's reservation fee.
setFee(fee: double)	Public	Void	Sets the room's reservation fee.

Table 46: SpecialRoom Class Table

2.2.2.4.7 User

The User class is an abstract class that represents the interface for all users. It shows the shared attributes of the abstract class including the user's ID, email, password, name, role, display name, birth date, profile image, and defines the shared payment, meeting-management, and complaint-filing behavior inherited by both the Client and Administrator classes.

Class Name	User		
Attribute	Type	Access	Description
userID	Int	Protected	Identifier unique to the specific user.
userEmail	String	Protected	User's email address.
passwordHash	String	Protected	User's hashed password.
lastName	String	Protected	User's last name.
firstName	String	Protected	User's first name.
role	String	Protected	User's account role (Client or Administrator).
displayName	String	Protected	Preferred name displayed for the user.
birthDate	String	Protected	User's date of birth.
profileImage	Byte[]	Protected	User's stored profile image.
failedAttempts	Int	Protected	Number of consecutive failed login attempts since the last successful login or reset.
lastFailed	LocalDateTime	Protected	Timestamp of the user's most recent failed login attempt.
lockedTimeTo	LocalDateTime	Protected	Timestamp the account remains locked until, after repeated failed login attempts.
Method	Visibility	Return Type	Description
User(userID: int, userEmail: String, passwordHash: String, firstName: String, lastName: String, role: String, displayName: String, birthDate: String, profileImage: byte[], failedAttempts: int, lastFailed: LocalDateTime, lockedTimeTo: LocalDateTime)	Protected	N/A	Initializes a new user with the given account details.
getUserID()	Public	Int	Returns the user's ID.
getUserEmail()	Public	String	Returns the user's email address.
getPasswordHash()	Public	String	Returns the user's hashed password

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

getLastName()	Public	String	Returns the user's last name
getFirstName()	Public	String	Returns the user's first name
payment()	Public	Void	Processes a payment.
manageMeeting()	Public	Void	Manages a meeting.
fileComplaint()	Public	Void	Files a complaint.
getRole()	Public	String	Returns the user's role.
getProfileImage()	Public	Byte[]	Returns the user's stored profile image.
getDisplayName()	Public	String	Returns the user's display name.
getBirthDate()	Public	String	Returns the user's date of birth.
getFailedAttempts()	Public	Int	Returns the user's count of consecutive failed login attempts.
getLastFailedLogin()	Public	LocalDateTime	Returns the timestamp of the user's most recent failed login attempt.
getLockedTimeTo()	Public	LocalDateTime	Returns the timestamp the user's account remains locked until.
setFailedAttempts(failedAttempts: int)	Public	Void	Sets the user's count of consecutive failed login attempts.
setLastFailedLogin(lastFailed: LocalDateTime)	Public	Void	Sets the timestamp of the user's most recent failed login attempt.
setLockedTimeTo(lockedTimeTo: LocalDateTime)	Public	Void	Sets the timestamp the user's account remains locked until.

Table 47: User Class Table

2.2.2.5 Repositories

The repository package contains the classes responsible for reading and writing persisted data using Spring's JdbcTemplate. The MeetingRepository and UserRepository classes execute the SQL statements used to create, retrieve, update, and delete Meeting and User records, while the ComplaintRowMapper, MeetingRowMapper, and UserRowMapper classes convert raw database rows into the corresponding Complaint, Meeting, or User object defined in the model package. The repository package is called directly by the controller package, and by the service package's MeetingService and UserService classes, to satisfy each request without exposing SQL to the rest of the application.

2.2.2.5.1 ComplaintRowMapper

The ComplaintRowMapper class converts a single row of the Complaints table into a Complaint object. It implements Spring's RowMapper interface and is used by the ComplaintController to translate query results into model objects.

Class Name	ComplaintRowMapper		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
mapRow(rs: ResultSet, rowNum: int)	Public	Complaint	Reads the given result set row and returns the corresponding Complaint object, restoring the ID, status, filing date, and administrator response.

Table 48: ComplaintRowMapper Class Table

2.2.2.5.2 MeetingRepository

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

The MeetingRepository class is the repository implementation for the Meeting module. It shows the main responsibilities of the class including creating, retrieving, updating, and deleting Meeting records in the database, and checking whether a room is available for a given time range.

Class Name	MeetingRepository		
Attribute	Type	Access	Description
jdbcTemplate	JdbcTemplate	Private	Reference used to execute meeting-related queries.
Method	Visibility	Return Type	Description
MeetingRepository(JdbcTemplate: JdbcTemplate)	Public	N/A	Initializes the repository with the given data-access dependency.
createMeeting(meeting: Meeting)	Public	int	Inserts the given meeting into the database.
findById(meetingID: int)	Public	Meeting	Returns the meeting matching the given ID.
findByCreator(userID: int)	Public	List<Meeting>	Returns the meetings created by the given user.
findAll()	Public	List<Meeting>	Returns every meeting in the database.
updateMeeting(meeting: Meeting)	Public	int	Updates the stored details of the given meeting.
deleteMeeting(meetingID: int)	Public	int	Deletes the meeting matching the given ID.
isRoomAvailable(roomId: int, startTime: LocalTime, endTime: LocalTime)	Public	Boolean	Returns whether the given room has no meeting that overlaps the given start and end time, used by MeetingService before scheduling a meeting.

Table 49: MeetingRepository Class Table

2.2.2.5.3 MeetingRowMapper

The MeetingRowMapper class converts a single row of the Meetings table into a Meeting object. It implements Spring's RowMapper interface and is used by the MeetingRepository and ScheduleController to translate query results into model objects.

Class Name	MeetingRowMapper		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
mapRow(rs: ResultSet, rowNum: int)	Public	Meeting	Reads the given result set row and returns the corresponding Meeting object, parsing its stored date, start time, and end time.
parseDate(value: String)	Private	LocalDate	Parses the given date string into a Date, or returns null if the value is blank.
parseTime(value: String)	Private	LocalTime	Parses the given time string into a Time, appending seconds if only hours and minutes were given, or returns null if the value is blank.

Table 50: MeetingRowMapper Table

2.2.2.5.4 UserRepository

The UserRepository class is the repository implementation for the User module. It shows the main responsibilities

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

for the class, including retrieving a user by email and updating a user's profile information (with an optional profile image), and recording, resetting, and clearing a user's failed-login and lockout state.

Class Name	UserRepository		
Attribute	Type	Access	Description
jdbcTemplate	JdbcTemplate	Private	Reference used to execute user-related queries.
Method	Visibility	Return Type	Description
UserRepository(jdbcTemplate: JdbcTemplate)	Public	N/A	Initializes the repository with the given data-access dependency.
findByEmail(email: String)	Public	User	Returns the user matching the given email address.
updateProfile(email: String, displayName: String, birthDate: String, image: byte[])	Public	Void	Updates the given user's display name, birth date, and profile image.
updateProfileWithoutImage(email: String, displayName: String, birthDate: String)	Public	Void	Updates the given user's display name and birth date, leaving their stored profile image unchanged.
updateLoginFailure(userId: long, failedAttempts: int, lastFailedLogin: LocalDateTime, lockedTimeTo: LocalDateTime)	Public	Void	Saves the given user's updated failed-login count, timestamp of the failure, and lock-expiry time.
resetLoginFailures(email: String)	Public	Void	Clears the given user's failed-login count and lock status after a successful login.
unlockAccount(email: String)	Public	Void	Clears the given user's failed-login count and lock status, releasing a locked account.

Table 51: UserRepository Class Table

2.2.2.5.5 UserRowMapper

The UserRowMapper class converts a single row of the Users table into either a Client or Administrator object, depending on the stored role. It implements Spring's RowMapper interface and is used throughout the repository and controller packages to translate query results into model objects.

Class Name	UserRowMapper		
Attribute	Type	Access	Description
N/A	N/A	N/A	N/A
Method	Visibility	Return Type	Description
mapRow(rs: ResultSet, rowNum: int)	Public	User	Reads the given result set row and returns a Client or Administrator object matching the row's stored role.

Table 52: UserRowMapper Class Table

2.2.2.6 Security

The security package contains the classes that respond to authentication and authorization events raised by the security filter chain configured in the config package. The LoginSuccessHandler and LoginFailureHandler classes route a user to the appropriate landing page or back to the login page depending on the results of a login attempt and, with the UserService, track repeated failed attempts to temporarily lock an account, the PmssAccessDeniedHandler class responds when an authenticated user attempts to access a resource they were not authorized for, and the

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

SecurityLogger class records each of these events. Every class in this package is registered by the SecurityConfig class in the config package and relies on it to be used.

2.2.2.6.1 LoginFailureHandler

The LoginFailureHandler class handles a failed login attempt. It implements Spring Security's AuthenticationFailureHandler interface, records the failed attempt through the SecurityLogger, distinguishes an attempt against an already-locked account from an ordinary failure, and redirects the user back to the login page with the corresponding error indicator.

Class Name	LoginFailureHandler		
Attribute	Type	Access	Description
securityLogger	SecurityLogger	Private	Reference used to record the failed login attempt.
userService	UserService	Private	Reference used to record the failed attempt against the user's account for lockout tracking.
Method	Visibility	Return Type	Description
LoginFailureHandler(securityLogger: SecurityLogger, userService: UserService)	Public	N/A	Initialize the handler with the given security logger and user service.
onAuthenticationFailure(request: HttpServletRequest, response: HttpServletResponse), exception: AuthenticationException)	Public	Void	Records the failed login attempt through the SecurityLogger. If the account is already locked, it redirects to the login page with a locked-account indicator. Otherwise, it records the failed attempt against the user's account through the UserService and redirects with a standard error indicator.

Table 53: LoginFailureHandler Class Table

2.2.2.6.2 LoginSuccessHandler

The LoginSuccessHandler class handles a successful login attempt. It implements Spring Security's AuthenticationSuccessHandler interface, clears any previously recorded failed-login attempts through the UserService, records the successful attempt through the SecurityLogger, and redirects the user to the landing page appropriate to their role.

Class Name	LoginSuccessHandler		
Attribute	Type	Access	Description
securityLogger	SecurityLogger	Private	Reference used to record the successful login attempt.
userService	UserService	Private	Reference used to clear the user's failed-login count on a successful login.
Method	Visibility	Return Type	Description
LoginSuccessHandler(securityLogger: SecurityLogger, userService: UserService)	Public	N/A	Initialize the handler with the given security logger and user service.
onAuthenticationSuccess(request: HttpServletRequest, response: HttpServletResponse), authentication: Authentication)	Public	Void	Clears the user's failed-login count through the UserService, records the successful login attempt through the SecurityLogger, and redirects the user to the Administrator or Client landing page based on their role.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Table 54: LoginSuccessHandler Class Table

2.2.2.6.3 PmssAccessDeniedHandler

The PmssAccessDeniedHandler class handles a request made by an authenticated user to a resource they are not authorized to access. It implements Spring Security's AccessDeniedHandler interface, records the failed authorization attempt through the SecurityLogger, and returns an access denied response.

Class Name	PmssAccessDenied		
Attribute	Type	Access	Description
securityLogger	SecurityLogger	Private	Reference used to record the failed authorization attempt.
Method	Visibility	Return Type	Description
PmssAccessDeniedHandler(securityLogger: SecurityLogger)	Public	N/A	Initializes the handler with the given security logger.
handle(request: HttpServletRequest, response: HttpServletResponse, exception: AccessDeniedException)	Public	Void	Records the failed authorization attempt and returns a forbidden response.

Table 55: PmssAccessDeniedHandler Class Table

2.2.2.6.4 SecurityLogger

The SecurityLogger class is the shared logging component for the security package. It shows the main responsibilities of the class, including recording login successes and failures, account lockouts, logouts, authorization failures, Client and Administrator account creation, room and meeting changes, billing updates, and complaint activity.

Class Name	SecurityLogger		
Attribute	Type	Access	Description
logger	Logger	Private	Logger used to write security events.
Method	Visibility	Return Type	Description
loginSuccess(email: String)	Public	Void	Records a successful login by the given user.
loginFailure(email: String)	Public	Void	Records a failed login attempt for the given user.
logout(email: String)	Public	Void	Records a logout by the given user.
authorizationFailure(email: String, path: String)	Public	Void	Records an authorization failure by the given user against the given path.
adminAccountCreated(creatorEmail: String, newAdminEmail: String)	Public	Void	Records the creation of a new administrator account and the administrator who created it.
clientAccountCreated(email: String)	Public	Void	Records the creation of a new Client account.
profileChanged(email: String)	Public	Void	Records a profile change made by the given user.
roomCreated(adminEmail: String, roomNumber: int)	Public	Void	Records the creation of a new room by the given administrator.
roomDeleted(adminEmail: String,	Public	Void	Records the deletion of a room by the given administrator.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

roomNumber: int)			
meetingCreated(email: String, meetingName: String)	Public	Void	Records the creation of a new meeting by the given user.
meetingDeleted(email: String, meetingID: int)	Public	Void	Records the deletion of a meeting by the given user.
attendeeAdded(email: String, meetingID: int, participantID: int)	Public	Void	Records that the given user added the given participant to the given meeting.
attendeeRemoved(email: String, meetingID: int, participantID: int)	Public	Void	Records that the given user removed the given participant from the given meeting.
billingUpdated(actorEmail: String, clientID: int)	Public	Void	Records that the given user updated the given client's billing information.
complaintFiled(email: String, meetingID: int)	Public	Void	Records that the given user filed a complaint concerning the given meeting.
complaintResolved(adminEmail: String, complaintID: int)	Public	Void	Records that the given administrator responded to and resolved the given complaint.
accountLocked(email: String)	Public	Void	Records the given user's account was locked for 15 minutes after repeated failed login attempts.

Table 56: SecurityLogger Class Table

2.2.2.7 Service

The service package contains the classes that implemented the business logic of the PMSS system between the controller and repository packages. The MeetingService class validates and coordinates meeting operations before delegating persistence to the MeetingRepository, including confirming a room's availability before scheduling a meeting, and the UserService class retrieves and updates User accounts through the UserRepository, tracks and enforces a temporary account lockout after repeated failed login attempts, while also implementing Spring Security's UserDetailsService interface so that the config package's authentication provider can authenticate against it directly.

2.2.2.7.1 MeetingService

The MeetingService class is the service implementation for the Meeting module. It shows the main responsibilities of the class including creating, retrieving, updating, and deleting meetings by delegating to the MeetingRepository, and scheduling a meeting only after confirming its room is available.

Class Name	MeetingService		
Attribute	Type	Access	Description
meetingRepository	MeetingRepository	Private	Reference used to persist and retrieve meeting records.
Method	Visibility	Return Type	Description
MeetingService(meetingRepository: MeetingRepository)	Public	N/A	Initializes the service with the given meeting repository.
creatingMeeting(meeting: Meeting)	Public	Boolean	Saves the given meeting and returns whether it was created successfully.
getMeeting(meetingID: int)	Public	Meeting	Returns the meeting matching the given ID.
getAllMeetings()	Public	List<Meeting>	Returns every meeting in the system.

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

getUserMeetings(userID: int)	Public	List<Meeting>	Returns the meeting created by the given user.
updateMeeting(meeting: Meeting)	Public	Boolean	Saves changes to the given meeting and returns whether the update succeeded.
deleteMeeting(meetingID: int)	Public	Boolean	Deletes the meeting matching the given ID and returns whether the deletion succeeded.
scheduleMeeting(meeting: Meeting)	Public	Void	Checks that the meeting's room is available for its room ID and time range through the MeetingRepository, throwing an IllegalStateException if it is already booked, then creates the meeting.

Table 57: MeetingService Class Table

2.2.2.7.2 UserService

The UserService class is the service implementation for the User module. It implements Spring Security's UserDetailsService interface so that it can be used directly by the authentication provider. It shows the main responsibilities of the class, including retrieving a user by email and validating the updating a user's profile information and profile image, and recording, tracking, and resetting failed login attempts to temporarily lock an account after repeated failures.

Class Name	UserService		
Attribute	Type	Access	Description
userRepository	UserRepository	Private	Reference used to persist and retrieve user records.
securityLogger	SecurityLogger	Private	Reference used to record account lockouts.
Method	Visibility	Return Type	Description
UserService(userRepository: UserRepository, securityLogger: SecurityLogger)	Public	N/A	Intializes the service with the given user repository.
loadUserByUsername(email: String)	Public	UserDetails	Retrieves the user matching the given email and, if their account is currently locked, throws a LockedException (automatically clearing the lock and failed-attempt count first if the lock period has already expired) then returns a UserDetails object created from the stored credentials and role, used during authentication.
findByEmail(email: String)	Public	User	Returns the user matching the given email address.
updateProfile(email: String, displayName: String, birthDate: String, image: MultipartFile)	Public	Void	Validates the format and dimensions of an uploaded profile image, then updates the user's display name, birth date, and a profile image when a valid image is provided.
recordFailedLogin(email: String)	Public	Void	Increments the given user's failed-login count, restarting the count if the previous

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

			failure was more than 30 minutes ago, and locks the account for 15 minutes and records the lockout through the SecurityLogger once three failures have occurred within that window.
resetLoginFailures(email: String)	Public	Void	Clears the given user's failed-login count and lock status through the UserRepository.

Table 58: UserService Class Table

2.3 Test Results

Several different testing approaches were used to evaluate PMSS from different perspectives. Unit testing was used via JUnit 5 to test individual methods and components of the application to make sure they worked correctly and produced the expected results. Network auditing and mapping was performed using Nmap to identify the host, open ports, and services running on the development machine. Vulnerability scanning was performed using Nessus to look for potential security weaknesses in the PMSS web application and its configuration. Using these different approaches helped test both the functionality and security of PMSS rather than relying on one type of testing alone.

2.3.1 Unit and Integration Testing

The PennStateSoft Meeting Scheduling System employs two major types of automated testing: unit and integration tests. For both, the Maven framework is used as the main driver of tests. For the unit tests, every class is tested individually using Junit5 as the testing framework and Mockito as the mocking library. For the integration tests, the Spring Boot application is instantiated and tests using real HTTP requests in real sample scenarios. Across both types of testing, a copy of the database is stored in memory to not have the tests affect the actual database being used on the production or development environments. Ultimately, there are 184 unit tests and 82 integration tests, adding up to 266 total tests, all of which are passing.

2.3.1.1 Unit Tests

There are 184 unit tests spread across 27 classes, roughly grouped into 7 categories, shown below in Table 59: Unit Tests Overview. The categories are configuration tests, controller tests, exception tests, model tests, repository tests, security tests, and service tests. The configuration tests are the foundational tests making sure that the underlying security of the application is functioning correctly. The controller tests are for ensuring each package's controller classes can handle receiving and sending requests between boundary and model classes. The exception tests ensure that the global exception handler is working correctly. The model tests make sure that the object data structures are working as intended and can respond correctly to the controllers. The repository tests verify that the Java Database Connectivity API is functioning correctly. The security tests are for authenticating event handlers and the security logger. Finally, the service tests verify that the business-logic is valid and that events can process correctly.

Category	Classes	Number of Tests
Configuration	SecurityConfigTest	2
Controller	BillingControllerTest, ComplaintControllerTest, MeetingControllerTest, RegisterControllerTest, ReportControllerTest, RoomControllerTest, ScheduleControllerTest, SimpleControllersTest	88
Exception	GlobalExceptionHandlerTest	3
Model	ComplaintTest, MeetingTest, RoomTest, SpecialRoomTest, UserTest	25
Repository	ComplaintRowMapperTest, MeetingRepositoryTest, MeetingRowMapperTest, UserRepositoryTest, UserRowMapperTest	27
Security	LoginFailureHandler, LoginSuccessHandlerTest, PmssAccessDeniedHandlerTest, SecurityLoggerTest	11
Service	MeetingServiceTest, UserServiceTest	28

Table 59: Unit Tests Overview

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

2.3.1.2 Integration Tests

There are 82 integration tests spread across 11 classes, shown below in Table 60: Integration Tests Overview. The integration tests function by booting up the application and testing with actual HTTP requests, testing the entire stack of the application. Similar to unit tests, an in-memory clone of the database is used so as to not affect the real database values.

Class	Tested feature	Number of Tests
AttendeeManagementIT	Adding and removing participants from meetings, checking double-booking	6
AuthenticationIT	Logging in, role redirects, URL authorization	8
BillingIT	Saving card details, masking card number, billing validation, administrator billing updates	9
ComplaintIT	Filing complaints, administrator list/respond/status	9
DatabaseSetupIT	Testing schema	2
MeetingLifecycleIT	Creating, validating, editing, and deleting meetings	11
ProfileIT	Viewing and editing profiles, image uploads	6
RegistrationIT	Client and administrator account registration, checking for duplicate accounts, input validation	8
ReportIT	Listing reports dashboard, spreadsheet exports	7
RoomAdminIT	Adding and deleting rooms	8
ScheduleAdminIT	Listing and filtering administrator schedule dashboard	7

Table 60: Integration Tests Overview

2.3.2 Fuzz Testing

Tools: Burp Suite Community Edition v2026.7.3

Target: PMSS – 192.168.1.166:8080

Testing Tool: Intruder

Test Types: XSS, SQL Injection, Input Validation

Parameters Tested: Meeting creation, complaint submission, profile modification, room creation, and account registration parameters

Payloads: Malicious, malformed, boundary, empty, and unexpected input values

Result: Testing completed successfully

2.3.3 Security Testing

2.3.3.1 Create Meeting

Test	Parameter	Payload Tested	Results	Conclusion
XSS	meetingName	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful XSS was observed
SQL Injection	meetingName	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful SQL injection was observed

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Input Validation	meetingName	(empty), A, Meeting, very long string – 100 characters long, 123456789, , <space>, <>, Test@#\$\$%^&*(), ' " ; --, <script>, ../../test	All payload requests returned HTTP Status 403	Malicious and invalid input requests were rejected with HTTP 403 responses; no evidence of successful input-validation bypass was observed.
Directory Traversal	meetingName	../../etc/passwd	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful directory traversal was observed
XSS	meetingDate	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful XSS was observed
SQL Injection	meetingDate	' OR '1'='1, “ OR "1"="1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful SQL injection was observed
Input Validation	meetingDate	2026-08-14, 2026-02-30, 2026-13-01, 2026-00-01, not-a-date, , (empty)	All payload requests returned HTTP Status 403	Malicious and invalid input requests were rejected with HTTP 403 responses; no evidence of successful input-validation bypass was observed.
Directory Traversal	meetingDate	../../etc/passwd	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful directory traversal was observed
XSS	startTime	<script>alert(1)</script>, "><script>alert(1)</script>, <img src=x	All payload	Malicious requests were

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

		onerror=alert(1)>, ">	requests returned HTTP Status 403	rejected; no successful XSS was observed
SQL Injection	startTime	' OR '1'=1, " OR "1"=1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful SQL injection was observed
Input Validation	startTime	10:00, 25:00, 10:60, not-a-time, , (empty)	All payload requests returned HTTP Status 403	Malicious and invalid input requests were rejected with HTTP 403 responses; no evidence of successful input-validation bypass was observed.
Directory Traversal	startTime	../../etc/passwd	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful directory traversal was observed
XSS	endTime	<script>alert(1)</script>, "><script>alert(1)</script>, , ">	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful XSS was observed
SQL Injection	endTime	' OR '1'=1, " OR "1"=1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful SQL injection was observed
Input Validation	endTime	11:00, 25:00, 09:00, not-a-time, , (empty)	All payload requests returned HTTP Status 403	Malicious and invalid input requests were rejected with HTTP 403 responses; no evidence of successful input-validation bypass was

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

				observed.
Directory Traversal	endTime	../../../../etc/passwd	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful directory traversal was observed
XSS	roomNumber	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful XSS was observed
SQL Injection	roomNumber	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful SQL injection was observed
Input Validation	roomNumber	101, 0, -1, 999999, abc, , (empty)	All payload requests returned HTTP Status 403	Malicious and invalid input requests were rejected with HTTP 403 responses; no evidence of successful input-validation bypass was observed.
Directory Traversal	roomNumber	../../../../etc/passwd	All payload requests returned HTTP Status 403	Malicious requests were rejected; no successful directory traversal was observed

Table 61: Create Meeting Test Results

2.3.3.2 Create Complaint

Test	Parameter	Payload Tested	Results	Conclusion
XSS	summary	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	All payload requests returned HTTP Status 302	Requests were accepted and redirected; no successful XSS execution was observed.

PennStateSoft Meeting Scheduling System			Version:	1.0
			Date:	08/08/2026
SQL Injection	summary	' OR '1'=1, " OR "1"=1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP Status 302	Requests were accepted and redirected; no successful SQL injection was observed.
Input Validation	summary	(empty), , A, 50-character string, numeric input, !@#\$\$%^&*(), < > " ' &Test@#\$\$%^&*(), ' " ; --, <script>, ../../test	Most payload requests returned HTTP 302; !@#\$\$%^&*() returned HTTP 500	Input validation testing identified an HTTP 500 response for the special-character payload. Further investigation is required to determine whether this represents an input-validation weakness or an application error.
Input Validation	meetingID	15, 0, -1, 999999, abc, 1.5, , (empty)	Valid/integer values returned HTTP 302; non-integer and empty values returned HTTP 400	Invalid non-integer and empty inputs were rejected with HTTP 400. Numeric boundary values were accepted and should be verified for appropriate application-level validation.
Input Validation	complaintOption	Meeting Scheduling, Invalid Option, (empty), , <script>alert(1)</script>, ' OR '1'=1	All payload requests returned HTTP 302	All inputs were accepted at the HTTP request level. Further verification is required to determine whether invalid options were accepted and stored or rejected by application-level validation.

Table 62: Create Complaint Test Results

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

2.3.3.3 Change Profile

Test	Parameter	Payload Tested	Results	Conclusion
XSS	displayName	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	All payload requests returned HTTP Status 302	Requests were accepted and redirected; no successful XSS execution was observed.
SQL Injection	displayName	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP 302	Requests were accepted and redirected; no successful SQL injection was observed.
Input Validation	displayName	(empty), , A, 50-character string – all A, 123456789, !@#%\$%^&*(), <> " ' &	All payload requests returned HTTP 302	All tested inputs were accepted at the HTTP request level. Further verification is required to determine whether application-level display-name restrictions are enforced.
XSS	birthDate	<script>alert(1)</script>, "><script>alert(1)</script>	All payload requests returned HTTP Status 302	Requests were accepted and redirected; no successful XSS execution was observed.
SQL Injection	birthDate	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP 302	Requests were accepted and redirected; no successful SQL injection was observed.
Input Validation	birthDate	(empty), , 1980-11-11, 2000-01-01, 2026-02-30, not-a-date, 1980/11/11, 11-11-1980, 0000-00-00, 9999-99-99	All payload requests returned HTTP 302	All tested inputs were accepted at the HTTP request level. Further verification is required to determine whether invalid

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

				dates are rejected by application-level validation.
--	--	--	--	---

Table 63: Change Profile Test Results

2.3.3.4 Create Room

Test	Parameter	Payload Tested	Results	Conclusion
XSS	roomNumber	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	XSS payloads returned 400	No successful XSS observed
SQL Injection	roomNumber	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	SQLi payloads returned 400	No successful SQL injection observed
Input Validation	roomNumber	(empty), , 0, -1, 1, 999999, abc, 1.5, 104.5, !@#%&^&*()	Non-numeric formats returned HTTP 400; numeric boundary values returned HTTP 302	Numeric/type validation observed
XSS	roomType	<script>alert(1)</script>, "><script>alert(1)</script>, "">	Payloads returned 302; application behavior verified separately	No successful XSS observed after application behavior was verified
SQL Injection	roomType	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	Payloads returned HTTP 302	No successful SQL injection observed after application behavior was verified
Input Validation	roomType	(empty), , REGULAR, INVALID, invalid, 123, ROOM, CONFERENCE, <script>	Invalid values returned 302	Application-level validation should be verified
XSS	fee	<script>alert(1)</script>, "><script>alert(1)</script>, "">	XSS payloads returned 400	No successful XSS observed
SQL Injection	fee	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	SQLi payloads returned 400	No successful SQL injection observed
Input Validation	fee	(empty), whitespace, 0, -1, 100, 100.00, 0.01, 999999.99, -100.00, abc, 1.5, 100.999, NaN, !@#%&^&*()	Some invalid/nonstandard values returned HTTP 302; non-numeric payloads returned HTTP 400	Application-level range validation should be verified

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Authorization/RBA C	/admin/rooms/ad d	Valid request using Client session	zReturned HTTP 403	Client denied access to admin endpoint
Authentication	/admin/rooms/ad d	Valid request without authenticated session	Request denied/redirected	Unauthenticated access denied

Table 64: Create Room Test Results

2.3.3.5 Create Administrator

Test	Parameter	Payload Tested	Results	Conclusion
XSS	firstName	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	Payload requests returned HTTP 302	No successful XSS observed
SQL Injection	firstName	' OR '1'=1, " OR "1"=1, ' UNION SELECT NULL--, admin'--	Payload requests returned HTTP 302	No successful SQL injection observed
Input Validation	firstName	(empty), , A, 50-character string, 123456789, !@#%&^*(), John123, < > ' ' &	Most requests returned 302; !@#%&^*() returned HTTP 500	Invalid/special-character input caused an application error; validation should be reviewed
XSS	lastName	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	Payload requests returned HTTP 302	No successful XSS observed
SQL Injection	lastName	' OR '1'=1, " OR "1"=1, ' UNION SELECT NULL--, admin'--	Payload requests returned HTTP 302	No successful SQL injection observed
Input Validation	lastName	(empty), , A, 50-character string, 123456789, !@#%&^*(), John123, < > ' ' &	Most requests returned 302; !@#%&^*() returned HTTP 500	Invalid/special-character input caused an application error; validation should be reviewed
XSS	email	<script>alert(1)</script> "><script>alert(1)</script>	All requests returned HTTP 302	No successful XSS observed
SQL Injection	email	' OR '1'=1, " OR "1"=1, ' UNION SELECT NULL--, admin'--	All requests	No successful SQL

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

n			returned HTTP 302	injection observed
Input Validation	email	(empty), , test, test@, test@example, test@example.com , test@pennstatesoft.com , TEST@PENNSATESOFT.COM , test@otherdomain.com , a@b.c, letsuseaveryveryveryveryveryveryveryverylongemailname@pennstatesoft.com	All requests returned HTTP 302	Input was accepted/rejected through application processing; email/domain validation should be verified from the resulting behavior
SQL Injection	password	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP 302	No successful SQL injection observed
Input Validation	password	(empty), , a, 123456, password, Password1, Password123!, 100-character string	All requests returned HTTP 302	Password input was processed; password-strength/length validation should be verified
Input Validation	birthdate	(empty), 1980-02-29, 2000-01-01, 2026-02-30, not-a-date, 1980/02/29, 02-29-1980, 0000-00-00, 9999-99-99	All requests returned HTTP 302	Date input was processed; application behavior should be verified for invalid formats and dates

Table 65: Create Administrator Test Results

2.3.3.6 Create Client

Test	Parameter	Payload Tested	Results	Conclusion
XSS	firstName	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	Payload requests returned HTTP 302	No successful XSS observed
SQL Injection	firstName	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	Payload requests returned HTTP 302	No successful SQL injection observed
Input	firstName	(empty), , A, 50-character string, 123456789,	Most	Invalid/speci

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Validation	e	!@#%^^&*(), John123, < > ' ' &	requests returned 302; !@#%^^&*() returned HTTP 500	al-character input caused an application error; validation should be reviewed
XSS	lastName	<script>alert(1)</script>, "><script>alert(1)</script>, , "">	Payload requests returned HTTP 302	No successful XSS observed
SQL Injection	lastName	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	Payload requests returned HTTP 302	No successful SQL injection observed
Input Validation	lastName	(empty), , A, 50-character string, 123456789, !@#%^^&*(), John123, < > ' ' &	Most requests returned 302; !@#%^^&*() returned HTTP 500	Invalid/special-character input caused an application error; validation should be reviewed
XSS	email	<script>alert(1)</script> "><script>alert(1)</script>	All requests returned HTTP 302	No successful XSS observed
SQL Injection	email	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	All requests returned HTTP 302	No successful SQL injection observed
Input Validation	email	(empty), , test, test@, test@example, test@example.com , test@pennstatesoft.com , TEST@PENNSATESOFT.COM , test@otherdomain.com , a@b.c , letsuseaveryveryveryveryveryveryveryverylongemailname@pennstatesoft.com	All requests returned HTTP 302	Input was accepted/rejected through application processing; email/domain validation should be verified from the resulting behavior
SQL Injection	password	' OR '1'='1, " OR "1"="1, ' UNION SELECT NULL--, admin'--	All payload requests returned HTTP 302	No successful SQL injection observed
Input Validation	password	(empty), , a, 123456, password, Password1, Password123!, 100-character string	All requests returned	Password input was processed;

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

			HTTP 302	password-strength/length validation should be verified
Input Validation	birthdate	(empty), 1980-02-29, 2000-01-01, 2026-02-30, not-a-date, 1980/02/29, 02-29-1980, 0000-00-00, 9999-99-99	All requests returned HTTP 302	Date input was processed; application behavior should be verified for invalid formats and dates

Table 66: Create Client Test Results

Overall Assessment: Burp Suite testing was completed to check how PMSS handled unexpected and invalid input. The issues identified during testing were reviewed and addressed where necessary. No remaining issues were found that would prevent the system from passing the test.

2.3.4 Network Auditing / Mapping

Tools: Nmap 7.911

Target: PMSS – 192.168.1.166 : 8080

Scan Start Date : 2026-08-13 21:12 -0400

Result: Scan completed successfully

Type: SYN Stealth Scan at 21:12

Completed Service scan at 21:12, 6.07s elapsed

Results

Nmap successfully discovered the PMSS host and identified 4 open TCP ports out of 1000 ports scanned. PMSS was identified as running on port 8080 using Apache Tomcat.

Topology

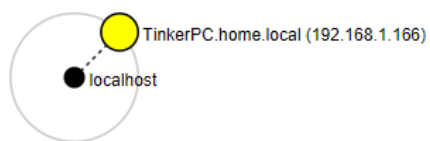


Figure 10: Network Mapping Topology

Port	State	Service	Version
135/tcp	Open	MSRPC	Microsoft Windows
139/tcp	Open	NetBios-SSN	Microsoft Windows NetBios-SSN
445/tcp	Open	Microsoft-DS	SMB
8080/tcp	Open	HTTP	Apache Tomcat - PMSS

Table 67: NMap Scan Results - Open Ports and Services

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Finding	Result	Assessment
PMSS HTTP Port	TCP 8080 Open	Expected
Web Server	Apache Tomcat	Expected
HTTP Methods	GET, HEAD, POST	Expected
Root URL	Redirects to /login	Expected
Host OS	Microsoft Windows 11	OS - Varies

Table 68: Nmap Scan Assessment – PMSS Application Findings

Overall Assessment: The Nmap scan successfully found the PMSS host and confirmed that the application is running on port 8080 through Apache Tomcat. The other open ports were identified as normal Windows services and are not part of the PMSS application. No other ports related to the PMSS were found during the scan.

2.3.5 Vulnerability Scanning

Tools: Nessus Home 10.12.3

Target: PMSS – 192.168.1.166 : 8080

Scan policy used : Web Application Tests

Safe checks : yes

Credentialed checks : no

Patch management checks : None

CGI scanning : enabled

Scan Start Date : 2026/8/13 14:55 Eastern Standard Time (UTC -04:00)

Scan duration : 895 sec

Result: Scan completed successfully

Security Weaknesses		
Findings	Result	Resolution
Web Server Transmits Cleartext Credentials	Login and registration credentials are transmitted over HTTP instead of HTTPS	Not Fixed - HTTPS is not configured because the submitted application is intended to run on as a local application rather than a server with a trusted TLS certificate
JSESSIONID Cookie Not Marked Secure	JSESSIONID has HttpOnly enabled but Secure is set to false	Fixed
Missing/Permissive CSP Frame-Ancestors	No restrictive frame-ancestors policy detected, but PMSS uses X-Frame-Options: DENY, which provides clickjacking protection	Allowed - Clickjacking protection is already present

Table 69: Security Weaknesses

Configuration and Low-Risk Findings		
Findings	Result	Resolution
Password Auto-Completion	Password fields on /login and /register do not explicitly disable autocomplete	Fixed
HTTP Methods Configuration	GET, HEAD, and OPTIONS are identified as allowed methods	Allowed – accepted due to low risk configuration finding
404 Behavior	Requests for nonexistent URLs receive a 302 redirect to /login instead of a 404 Not Found response	Allowed – accepted due to low risk configuration finding

Table 70: Configuration and Low-Risk Findings

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

Information Findings	
Findings	Result
Web Mirroring	Discovered /login and /register and their POST parameters
CGI Parameter Detection	The password parameters were identified as potentially sensitive.
CGI Generic Tests Load Estimation	Estimated the number of requests that could be generated by generic application tests as an estimate of 1746
Web Application Sitemap	Identified /login and /register as linkable application content
HTTP Redirect Information	The root URL / redirects to /login using HTTP 302
HTTP Information	Identified HTTP/1.1, no SSL/TLS, Keep-Alive, and the HTTP methods supported by the server

Table 71: Information Findings

Overall Assessment: The Nessus scan completed successfully and found several security and configuration issues in PMSS. The most important issues were that login and registration passwords were being sent over unencrypted HTTP and that the JSESSIONID cookie was not marked as Secure. Nessus also found that the password fields allowed autocomplete. These issues will be fixed and the application will be scanned again to make sure the fixes worked. The other findings were reviewed and determined to be low-risk or informational and do not currently require changes.

2.3.6 Bugs

Throughout development we tested the PennStateSoft Meeting Scheduling System with a combination of tools: static code analysis, unit testing, integration testing, fuzz testing, network auditing and mapping, and vulnerability scanning. The list of bugs and their resolution can be found below in Table 72: Bugs Found and Resolved.

2.3.6.1 Resolving Bugs

Test	Bugs / Issues Found	Resolution
Static Code Analysis Tool	Code quality and maintainability issues identified via SonarQube. 59 Issues found, 55 Resolved	Most issues were corrected through code changes. The remaining issues were reviewed and documented as allowed with the corresponding reasons for no changes made.
Unit Testing	New admin emails rejected as duplicates while duplicates accepted, user classes passing birthDate and displayName in swapped order, and meeting room ID always set to 0.	Negated unique email guard statement, reordered constructor arguments, and fixed room's ID initialization.
Integration Tests	ComplaintRowMapper read dateFiled column incorrectly causing 500 error, ComplaintController set validation errors but never set return value, user is redirected to incorrect URL after 3 invalid login attempts.	ComplaintRowMapper reads dateFiled column correctly, ComplaintController given return values, lockout redirection set to correct URL.
Fuzz Test	XSS, SQL injection, and input-validation payloads were tested against relevant PMSS parameters. No successful attacks or vulnerabilities requiring remediation were identified.	No changes required. Testing results were reviewed and documented; all tested malicious or invalid requests were rejected or handled without evidence of a successful vulnerability.
Network Auditing / Mapping	Open ports and services identified	Reviewed the results and determined which pers were related

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

		to PMSS and which were related to Windows services
Vulnerability Scanning	Security issues including the JSESSIONID cookie, password autocomplete, and HTTP credential transmissions	JSESSION and password autocomplete issues were resolved via applications.properties and the assigned HTML template. The HTTP/HTTPS issue was documented because PMSS is currently running in a local environment

Table 72: Bugs Found and Resolved

Final Security Review

The final security review was completed after all of the coding and testing was completed. The results from the different tests were reviewed to determine the overall security of PMSS. This included unit tests, integration tests, fuzz tests, network auditing / mapping, vulnerability scanning, and static code analysis.

2.4 Passed

The tests that passed showed that the system was working as expected and that no major security problems were found. Bugs and security issues that were found during the testing were fixed where possible and then tested again to ensure the fixed code worked as intended.

2.5 Passed with Recommendations

Some tests passed but had recommendations for further improvements. These did not cause the system to fail the security review, but making these changes in the future could improve the security, reliability, or maintainability of PMSS. These recommendations have been documented with the related test results.

2.6 Failed

No major security issues were found that would cause PMSS to fail the final security review. Issues found during testing were reviewed and fixed when possible. Any remaining issues or recommendations are documented in the individual testing sections.

2.7 Conclusion

PMSS passed with recommendations. The major issues found during coding and testing were addressed, while the remaining recommendations can be considered for future improvements to the system.

3. Team Member Log Sheets

3.1 Samuel Shumake

Date	Task	Duration
8/01/2026	Worked on entity classes, cleaning up codebase	02h 00m
8/04/2026	Investigating AWS possibility, working on Maven integration	01h 30m
8/06/2026	Learning Spring Boot framework	01h 00m
8/08/2026	Working on Schedule and Register modules	03h 00m
8/11/2026	Meeting, final implementation sweep on code	04h 00m
8/12/2026	Final implementation sweep and unit tests	02h 00m
8/13/2026	Integration tests and adding tests to final report	02h 00m
8/14/2026	Working on report and project demo	06h 00m
Total :		21h 00m

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

3.2 Victoria Worthington

Date	Task	Duration
7/31/26	Start PMSS in Java on IntelliJ, SQLite implementation, added all classes and attributes	02h 15m
8/4/26	Redesigned to Web-Based system and upload to GitHub adding teammates as collaborators	03h 0m
8/5/26	Profile.html, Spring Security	0h 45m
8/6/26	Fixed login.html	0h 5m
8/7/26	Profile-edit.html added to github	0h 20m
8/8/26	Documentation, sections 1, 1.1, 1.3	0h 20m
8/10/26	SQL injection mitigation	0h 35m
8/11/26	Security Audit Logging and debugging, Session Security, Testing Lists and Tools, profile image input, Start of Class Diagram, Group meeting	3h 0m
8/11/2026	Meeting with team members.	0h 30m
8/12/26	Login failure tracking, 15 minute lockout, error messages, registration input validation, password requirements, XSS prevention, race condition preventions, used SonarQube as code static analysis tool for PMSS	4h 15m
8/13/26	Section 2.1 - all charts completed (55/59 issues fixed). Section 2.5.1.4 Vulnerability Scanning via Nessus, Section 2.5.1.3 Network Auditing / Mapping, Section 2.5.2 Bugs table	5h 40m
8/14/26	Section 2.5.1.2 Fuzz Testing via Burp Suit Community Edition, Section 2.5.2 - Added Fuzz Testing bugs and resolutions, Section 1.2, 1.3, 1.4, Final Security Review	4h 0m
Total :		24h 45m

3.3 Lianjie Sun

Date	Task	Duration
08/01/2026	Start to familiar with the entire design of PMSS, and configured the project on IntelliJ with all required features, such as SQLite implementation.	2h 0m
08/02/2026	Prepare to coding Meeting section, added all pre-defined attributes and methods to the project.	3h 0m
08/03/2026	Work on html sections as the first draft.	3h 0m
08/04/2026	Complete the most of methods.	3h 0m
08/05/2026	Test all pre-defined security issues and potential bugs, there were several of errors detected.	3h 0m
08/06/2026	According to the principles of project functions, start to fix all detected errors/bugs and adjust incorrect logistic.	3h 0m
08/09/2026	Fixed all detected errors/bugs, and uploaded the latest version to GitHub for further discussion with team members.	2h 0m
08/11/2026	Meeting with team members.	0h 30m
08/12/2026	Work on project documents.	2h 0m
Total :		21h 30m

PennStateSoft Meeting Scheduling System	Version: 1.0
	Date: 08/08/2026

3.4 Tiffany Hart

Date	Task	Duration
8/6/26	Added site banner, developed Complaint system, site navigation, updated overall site appearance for the login, landing, complaint, and profile pages.	4hr 0m
8/11/2026	Meeting with team members.	0h 30m
8/11/26, 8/12/26, 8/13/26	Section 2.4	19hr 0m
8/13/26	Update Section 2.4	2hr 15m
8/13/26	Document formatting	0hr 30m
8/12/26, 8/13/26, 8/14/26	Class Diagrams	5hr 0m
Total :		31h 00m